

**UNIVERSITAS GUNADARMA
FAKULTAS TEKNOLOGI INDUSTRI**



PENULISAN ILMIAH

**PENGEMBANGAN SISTEM DETEKSI KESENJANGAN
KOMPETENSI DAN REKOMENDASI *SKILL*
MENGUNAKAN PENDEKATAN *MACHINE LEARNING*
PADA BIDANG IT**

**Nama : Annisa Ismidabilah
NPM : 50423191
Program Studi : Informatika
Pembimbing : Dr. Antonius Angga Kurniawan, ST., MMSI**

**Diajukan Guna Melengkapi Sebagian Syarat Dalam Menggapai Gelar Setara
Sarjana Muda**

**JAKARTA
2026**

PERNYATAAN ORISINALITAS DAN PUBLIKASI

Saya yang bertanda tangan di bawah ini,

Nama : Annisa Ismidabilah
NPM : 50423191
Judul PI : PENGEMBANGAN SISTEM DETEKSI
KESENJANGAN KOMPETENSI DAN REKOMENDASI
SKILL MENGGUNAKAN PENDEKATAN *MACHINE*
LEARNING PADA BIDANG IT
Tanggal Sidang :
Tanggal Lulus :

Menyatakan bahwa tulisan ini adalah merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apa pun telah mengikuti kaidah, etika yang berlaku. Mengenai isi dan tulisan adalah merupakan tanggung jawab Penulis, bukan Universitas Gunadarma. Demikian pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.



Jakarta, Agustus 2026


(Annisa Ismidabilah)

LEMBAR PENGESAHAN

Judul PI : PENGEMBANGAN SISTEM DETEKSI
KESENJANGAN KOMPETENSI DAN REKOMENDASI
SKILL MENGGUNAKAN PENDEKATAN *MACHINE*
LEARNING PADA BIDANG IT

Nama : Annisa Ismidabilah
NPM : 50423191
Program Studi : Informatika
Tanggal Sidang :
Tanggal Lulus :

Menyetujui

Pembimbing

Kasubbag, Sidang PI

(Dr. Antonius Angga Kurniawan, ST., MMSI) (Dr. Achmad Fahrurozi, S.Si., M.Si.)

Ketua Program Studi

(Prof. Dr. Lintang Yuniar Banowosari, S.Kom., M.Sc)

ABSTRAK

Penulisan Ilmiah. Informatika. Fakultas Teknologi Industri. Universitas Gunadarma. Tahun 2026

Kata kunci: klasifikasi *multi-label*, *machine learning*, *natural language processing*, sistem rekomendasi, *skill gap* Annisa Ismidabilah. 50423191

PENGEMBANGAN SISTEM DETEKSI KESENJANGAN KOMPETENSI DAN REKOMENDASI *SKILL* MENGGUNAKAN PENDEKATAN *MACHINE LEARNING* PADA BIDANG IT

(xii + 63 + Lampiran)

Penelitian ini bertujuan mengembangkan sistem berbasis web bernama ElvIT yang mampu mendeteksi kesenjangan kompetensi (*skill gap*) teknis di bidang Teknologi Informasi serta memberikan rekomendasi *skill* secara personal menggunakan pendekatan Natural Language Processing (NLP) dan Machine Learning. Sistem menerima input kompetensi melalui tiga jalur, yaitu pemindaian Curriculum Vitae (CV) berformat PDF, kuesioner self assessment, dan input manual, kemudian membandingkannya dengan benchmark kebutuhan *skill* industri yang dibangun dari dua dataset lowongan kerja IT, yaitu dataset JD2Skills yang diunduh dari portal lowongan kerja mycareersfuture.sg dalam bentuk berkas JSON mentah berisi 20.298 lowongan dan dataset anotasi NER berisi 418.870 token. Metodologi penelitian mengikuti kerangka CRISP-DM (Cross-Industry Standard Process for Data Mining). Model machine learning multi-label dibangun untuk memprediksi kebutuhan *skill* (top-N *skill*) pada 24 kategori pekerjaan, dengan input berupa 274 fitur hasil gabungan TF-IDF dan one-hot encoding, serta 250 kandidat *skill* sebagai label. Algoritma yang dibandingkan meliputi OneVsRest Logistic Regression, OneVsRest SGDClassifier, dan Random Forest (Multi-Output). Hasil evaluasi menggunakan metrik top-k menunjukkan bahwa OneVsRest Logistic Regression mencapai performa terbaik dengan F1@10 sebesar 0,6059 (Precision@10 0,8508 dan Recall@10 0,4705) sehingga dijadikan model terpilih. Keluaran sistem mencakup matched *skills*, gap *skills*, gap score, match score, dan rekomendasi *skill* prioritas yang divisualisasikan melalui radar chart dan bar chart.

Daftar Pustaka (2000 - 2026)

ABSTRACT

Annisa Ismidabilah. 50423191

DEVELOPMENT OF A SYSTEM FOR DETECTING SKILL GAPS AND PROVIDING SKILL RECOMMENDATIONS USING A MACHINE LEARNING APPROACH IN THE FIELD OF IT

Scientific Paper. Department of Informatics. Faculty of Industrial Technology. Gunadarma University. 2026

Keywords: machine learning, multi-label classification, natural language processing, recommendation system, skill gap

(xii + 63 + Appendix)

This study aims to develop a web-based system called ElvIT to detect technical skill gaps in the field of Information Technology and provide personalized skill recommendations using Natural Language Processing (NLP) and Machine Learning. The system receives competency input through three channels: PDF Curriculum Vitae (CV) scanning, self-assessment questionnaires, and manual input, then compares them against a job-skill benchmark built from two IT job datasets: the JD2Skills dataset downloaded from the mycareersfuture.sg job portal (Bhola et al., 2020) as raw JSON containing 20,298 job postings, and an NER annotation dataset (418,870 tokens). The methodology follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) framework. A multi-label machine learning model predicts required skills (top-N skills) for 24 job categories, using 274 features from TF-IDF and one-hot encoding as input, with 250 candidate skills as labels. The compared algorithms are OneVsRest Logistic Regression, OneVsRest SGDClassifier, and Random Forest (Multi-Output). Evaluation using top-k metrics shows that OneVsRest Logistic Regression achieves the best performance with F1@10 of 0.6059 (Precision@10 0.8508 and Recall@10 0.4705), thus selected as the final model. System outputs include matched skills, gap skills, gap score, match score, and prioritized skill recommendations visualized through radar and bar charts.

References (2000 - 2026)

KATA PENGANTAR

Segala puji dan syukur ke hadirat Tuhan Yang Maha Kuasa yang telah memberikan berkat, anugerah dan karunia yang melimpah, sehingga penulis dapat menyelesaikan Penulisan Ilmiah ini. Penulisan Ilmiah ini disusun guna melengkapi sebagian syarat dalam mencapai gelar Setara Sarjana Muda pada Program Studi Informatika, Fakultas Teknologi Industri, Universitas Gunadarma. Adapun judul Penulisan Ilmiah ini adalah “Pengembangan Sistem Deteksi Kesenjangan Kompetensi dan Rekomendasi *Skill* Menggunakan Pendekatan *Machine Learning* pada Bidang IT”.

Walaupun banyak kesulitan yang penulis harus hadapi ketika menyusun Penulisan Ilmiah ini, namun berkat bantuan dan dorongan dari berbagai pihak akhirnya tugas ini dapat diselesaikan dengan baik. Untuk itu penulis mengucapkan terima kasih, kepada:

1. Prof. Dr. E.S. Margianti, SE., MM., selaku Rektor Universitas Gunadarma.
2. Prof. Dr. -Ing. Adang Suhandra, SSi., S.Kom., MSc., selaku Dekan Fakultas Teknologi Industri Universitas Gunadarma.
3. Prof. Dr. Lintang Yuniar Banowosari, S.Kom., M.Sc., selaku Ketua Program Studi Informatika Universitas Gunadarma.
4. Dr. Achmad Fahrurrozi, S.Si, M.Si., selaku Kepala Sub Bagian Sidang Penulisan Ilmiah Fakultas Teknologi Industri Universitas Gunadarma.
5. Dr. Antonius Angga Kurniawan, ST., MMSI, selaku Dosen Pembimbing yang telah memberikan bimbingan, arahan, dan saran yang sangat berharga selama proses penelitian dan penulisan berlangsung.

6. Donny Hidayat dan Eka Patmasari, selaku orang tua penulis yang senantiasa memberikan doa, dukungan, dan motivasi selama penulis menjalani masa studi.
7. Muhammad Daffa' Alfiannoor, selaku orang yang senantiasa memberikan dukungannya, menemani, memberikan semangat serta motivasi selama proses penelitian dan penulisan berlangsung.

Jakarta, 11 Agustus 2026



(Annisa Ismidabilah)

DAFTAR ISI

PENULISAN ILMIAH	i
PERNYATAAN ORISINALITAS DAN PUBLIKASI.....	ii
LEMBAR PENGESAHAN	iii
ABSTRAK	iv
ABSTRACT	v
KATA PENGANTAR.....	vi
DAFTAR ISI.....	viii
DAFTAR TABEL	xi
DAFTAR GAMBAR.....	xii
DAFTAR LAMPIRAN	xiii
1. PENDAHULUAN.....	1
1.1. Latar Belakang	1
1.2. Ruang Lingkup	4
1.3. Tujuan Penelitian.....	5
1.4. Metode Penelitian.....	6
1.5 Sistematika Penulisan.....	8
2. Tinjauan Pustaka	10
2.1. Kesenjangan Kompetensi (<i>Skill Gap</i>)	10
2.2. Standar Kompetensi Kerja.....	10
2.2.1. SKKNI (Standar Kompetensi Kerja Nasional Indonesia)	11
2.2.2. SFIA (<i>Skills Framework for the Information Age</i>).....	11
2.3. <i>Natural Language Processing (NLP)</i>	12
2.3.1. <i>Text Preprocessing</i>	12
2.3.2. Ekstraksi Teks PDF dan <i>Named Entity Recognition</i>	13
2.4. Ekstraksi Fitur Teks.....	13
2.4.1. TF-IDF (Term Frequency-Inverse Document Frequency)	14
2.4.2. <i>One-Hot Encoding</i>	14
2.5. Klasifikasi <i>Multi-Label</i>	14
2.5.1. <i>Binary Relevance</i> dan <i>OneVsRest Classifier</i>	15

2.5.2. <i>Logistic Regression</i>	15
2.5.3. <i>Stochastic Gradient Descent Classifier (SGDClassifier)</i>	16
2.5.4. <i>Random Forest</i>	16
2.6. Sistem Rekomendasi <i>Skill</i>	17
2.7. CRISP-DM (Cross-Industry Standard Process for Data Mining)	18
2.8. Evaluasi Model.....	18
2.8.1. Metrik Top-K (<i>Precision@k, Recall@k, F1@k</i>).....	19
2.8.2. <i>Hamming Loss</i>	19
2.8.3. <i>Confusion Matrix Multi-Label</i>	19
2.8.4. Kurva ROC dan <i>Precision-Recall</i>	20
2.9. Penelitian Terdahulu.....	20
3. PEMBAHASAN DAN IMPLEMENTASI	23
3.1 Gambaran Umum Penelitian	23
3.2 Business Understanding	24
3.3 Data Understanding	25
3.4 Data Preparation	29
3.4.1. Pembersihan Data	30
3.4.2. Normalisasi Skill.....	31
3.4.3. Pembentukan Kandidat Skill (Label).....	31
3.4.4. Pembentukan Fitur TF-IDF	32
3.4.5. Encoding Kategori Pekerjaan (One-Hot Encoding)	32
3.4.6. Train-Test Split.....	32
3.5 Modeling	33
3.5.1. OneVsRest Logistic Regression	34
3.5.2. OneVsRest SGDClassifier.....	35
3.5.3. Random Forest Multi-Output.....	36
3.5.4. Proses Pelatihan	38
3.6 Evaluation.....	38
3.6.1. Validasi Silang (K-Fold Cross Validation).....	39
3.6.2. Hasil Evaluasi Ketiga Model	40
3.6.3. Analisis Perbandingan Ketiga Model	43
3.6.5. Komponen Model Final	45

3.6.4. Evaluasi Lanjutan pada Model Final	45
3.7 Deployment	48
3.7.1 Perancangan Sistem	49
3.7.2 Komponen Deployment	52
3.7.3 Persiapan Komponen Deployment	53
3.7.4 Deployment Backend	54
3.7.5 Deployment Frontend	55
3.7.6 Pengujian Sistem	56
4. PENUTUP	58
4.1 Kesimpulan	58
4.2 Saran	59
DAFTAR PUSTAKA	60
LAMPIRAN	L-1

DAFTAR TABEL

Tabel 3.1 Tahapan Penelitian Berdasarkan CRISP-DM.....	23
Tabel 3.2 Tujuan dan Keluaran Sistem ElvIT.....	25
Tabel 3.3 Ringkasan Dataset Lowongan Kerja IT	26
Tabel 3.4 Tahapan Data Preparation.....	30
Tabel 3.5 Contoh Hasil Pembersihan Data	31
Tabel 3.6 Konfigurasi Fitur Model	33
Tabel 3.7 Model Multi-Label yang Dibandingkan	34
Tabel 3.8 Parameter OneVsRest Logistic Regression	34
Tabel 3.9 Parameter OneVsRest SGDClassifier.....	36
Tabel 3.10 Parameter Random Forest Multi-Output.....	37
Tabel 3.11 Metrik Evaluasi Model.....	38
Tabel 3.12 Perbandingan Hasil Evaluasi Ketiga Model	40
Tabel 3.13 Nilai Evaluasi Model Final (OneVsRest Logistic Regression)	43
Tabel 3. 14 Komponen Model Final	45
Tabel 3. 15 Pengujian Endpoint API ElvIT	56
Tabel 3. 16 Pengujian Fitur Scan CV	57

DAFTAR GAMBAR

Gambar 3.1 Distribusi Posting per Job Category	27
Gambar 3.2 Distribusi Jumlah Skill yang Dibutuhkan per Job Category	28
Gambar 3.3 Top 20 Skills yang Paling Dibutuhkan Industri	29
Gambar 3.4 Arsitektur Model OneVsRest Logistic Regression / SGDClassifier.	35
Gambar 3.5 Arsitektur Model Random Forest Multi-Output	37
Gambar 3.6 K-Fold Cross Validation Model	39
Gambar 3.7 Perbandingan Precision, Recall, dan F1@10 Antar Model	42
Gambar 3.8 Perbandingan Precision-Recall Antar Model	42
Gambar 3.9 Precision-Recall Kurva per Nilai K	42
Gambar 3.10 Confusion Matrix Multi-Label	46
Gambar 3. 11 Classification Report per Skill	46
Gambar 3. 12 ROC-PR Curve Model Terbaik	47
Gambar 3. 13 Average Precision@10 per Kategori	47
Gambar 3. 14 Feature Importance Semua Model	48
Gambar 3.15 Workflow Diagram Sistem ElvIT	49
Gambar 3.16 Use Case Diagram Sistem ElvIT	50
Gambar 3.17 Activity Diagram Sistem ElvIT	51
Gambar 3.18 Arsitektur Deployment Sistem ElvIT	53
Gambar 3.19 Hasil Uji Menggunakan npm run build	57

DAFTAR LAMPIRAN

Lampiran 1 Listing Program – API Backend (main.py)	1
Lampiran 2 Listing Program Schemas API (schemas.py)	5
Lampiran 3 Listing Program Model Service (model_service.py).....	6
Lampiran 4 Listing Program – CV Service / NLP Pipeline (cv_service.py)	9
Lampiran 5 Listing Program – Data Service (data_service.py)	15
Lampiran 6 Output Aplikasi Elvit.....	15

1. PENDAHULUAN

1.1. Latar Belakang

Transformasi digital yang terjadi secara global telah mendorong perubahan signifikan pada kebutuhan tenaga kerja di bidang teknologi informasi. *World Economic Forum* memproyeksikan bahwa pada tahun 2025 sekitar 85 juta pekerjaan akan tergantikan oleh otomatisasi, sementara 97 juta pekerjaan baru akan muncul dengan dominasi pada bidang teknologi seperti kecerdasan buatan, komputasi awan, keamanan siber, dan rekayasa data (*The Future of Jobs Report 2020* | *World Economic Forum*, 2020). Kondisi ini menunjukkan bahwa kebutuhan terhadap kompetensi di bidang teknologi tidak hanya meningkat, tetapi juga terus berubah secara dinamis mengikuti perkembangan inovasi, sehingga identifikasi kompetensi yang akurat dan berkelanjutan menjadi kebutuhan mendesak baik bagi industri maupun tenaga kerja itu sendiri.

Fenomena tersebut turut dirasakan di Indonesia. Percepatan transformasi digital di Indonesia didukung melalui berbagai kebijakan strategis seperti *Visi Indonesia Digital 2045* (Kementerian Komunikasi dan Informatika Republik Indonesia, 2023) dan *Making Indonesia 4.0* (Kementerian Perindustrian Republik Indonesia, 2018), yang menegaskan bahwa penguasaan teknologi informasi menjadi faktor utama dalam meningkatkan daya saing nasional. Dampaknya, profesional di bidang IT dituntut untuk terus meningkatkan kompetensi dan mampu beradaptasi terhadap kebutuhan industri yang berkembang secara cepat. Pergeseran kebutuhan ini juga terlihat pada skala industri IT secara umum, di mana kombinasi *soft skill* dan *hard skill* yang diminta pasar kerja terus berubah mengikuti tren teknologi terkini sehingga standar kompetensi yang statis cenderung cepat usang (Ternikov, 2022).

Kondisi di atas memunculkan permasalahan mendasar berupa kesenjangan kompetensi (*skill gap*), yaitu ketidaksesuaian antara kemampuan yang dimiliki tenaga kerja dengan kompetensi yang dibutuhkan oleh industri. Kondisi ini

menyebabkan rendahnya kesiapan tenaga kerja dalam memenuhi kebutuhan pasar serta berdampak pada menurunnya produktivitas organisasi (Framesthi et al., 2025). Sayangnya, proses identifikasi *skill gap* saat ini masih banyak dilakukan secara manual melalui wawancara, kuesioner, maupun penilaian subjektif, yang dinilai kurang efektif karena memerlukan waktu lama, tidak skalabel, serta berpotensi menghasilkan bias dalam pengambilan keputusan.

Di sisi lain, tersedia berbagai sumber data yang berpotensi digunakan untuk mengidentifikasi kompetensi secara lebih objektif, seperti profil profesional, *curriculum vitae*, hasil asesmen, dan deskripsi lowongan pekerjaan (Elia et al., 2023). Namun, pemanfaatan data tersebut masih belum optimal. Akibatnya, organisasi mengalami kesulitan dalam merancang program pengembangan kompetensi yang tepat sasaran (Braun et al., 2025), sementara individu tidak memiliki panduan yang jelas terkait keterampilan yang perlu ditingkatkan (Cedefop, 2018).

Perkembangan teknologi *machine learning* dan *natural language processing* (NLP) memberikan peluang untuk mengatasi permasalahan tersebut secara lebih efektif dan objektif. Sebuah tinjauan komprehensif terhadap metode identifikasi *skill* dari iklan lowongan kerja daring menunjukkan bahwa pendekatan berbasis NLP semakin banyak digunakan untuk mengekstraksi kompetensi secara otomatis dari data teks tidak terstruktur, menggantikan pendekatan berbasis pencocokan kata kunci yang rentan terhadap ambiguitas bahasa (Khaouja et al., 2021). Pada level ekstraksi kompetensi dari dokumen seperti *resume* dan lowongan kerja, pendekatan klasifikasi *multi-label* mulai banyak dikembangkan. Melalui arsitektur *convolutional neural network* (CNN), sejumlah peneliti berhasil memprediksi keterampilan tingkat tinggi dari *resume* meskipun tidak disebutkan secara eksplisit di dalam dokumen tersebut (Jiechieu & Tsopze, 2020). Pendekatan *multi-label* serupa juga diterapkan pada data lowongan kerja berbahasa Belanda menggunakan model bahasa RobBERT untuk mengekstraksi *skill* implisit maupun eksplisit dalam skema *extreme multi-label classification* (Vermeer, 2022), sementara pendekatan berbasis *graph neural network* dikembangkan untuk mempelajari relasi antara pekerjaan dan *skill* pada skala besar guna memprediksi

skill yang belum tercantum secara eksplisit dalam deskripsi pekerjaan (Goyal et al., 2023).

Meskipun demikian, penelitian-penelitian tersebut umumnya masih terbatas pada satu aspek, yaitu berfokus pada ekstraksi atau prediksi kebutuhan *skill* semata, tanpa dilanjutkan ke tahap deteksi kesenjangan kompetensi individu maupun rekomendasi pengembangannya. Di sisi lain, penelitian mengenai sistem rekomendasi pembelajaran umumnya dikembangkan secara terpisah dari analisis *skill gap*. Sebagai contoh, pendekatan *collaborative filtering* berbasis *K-Nearest Neighbor*, *Singular Value Decomposition*, dan model berbasis jaringan saraf telah digunakan untuk merekomendasikan kursus pembelajaran daring kepada pengguna berdasarkan pola preferensi historisnya (Jena et al., 2023), namun tanpa mengintegrasikan hasil analisis kesenjangan kompetensi secara otomatis sebagai dasar rekomendasi. Selain itu, sebagian besar penelitian di atas dilakukan dalam konteks pasar kerja dan bahasa negara atau kawasan tertentu, seperti Belanda dan kawasan *Commonwealth of Independent States*, sehingga belum tentu sesuai dengan karakteristik struktur okupasi dan istilah kompetensi pada industri IT di Indonesia.

Kondisi ini menunjukkan adanya kebutuhan akan suatu pendekatan yang mampu mengintegrasikan prediksi kebutuhan *skill*, deteksi kesenjangan kompetensi, dan rekomendasi pengembangan keterampilan dalam satu sistem yang komprehensif dan kontekstual bagi industri IT di Indonesia.

Berdasarkan permasalahan tersebut, penelitian ini bertujuan untuk mengembangkan sistem terintegrasi yang mampu memprediksi kebutuhan *skill* pada suatu kategori pekerjaan IT menggunakan pendekatan klasifikasi *multi-label*, mendeteksi kesenjangan antara *skill* yang dimiliki pengguna dengan kebutuhan industri, serta memberikan rekomendasi pengembangan keterampilan secara personal. Sistem menerima *input* kompetensi melalui tiga jalur, yaitu pemindaian *curriculum vitae*, *self assessment*, dan *input* manual, kemudian membandingkannya dengan kebutuhan *skill* industri yang diperoleh dari data lowongan kerja. Metodologi penelitian mengikuti kerangka kerja CRISP-DM (*Cross-Industry Standard Process for Data Mining*) dengan mengacu pada standar kompetensi

nasional seperti SKKNI (Standar Kompetensi Kerja Nasional Indonesia) serta kerangka SFIA (*Skills Framework for the Information Age*). Sistem yang diusulkan diharapkan dapat meningkatkan efektivitas proses pengembangan sumber daya manusia di bidang teknologi informasi, serta memberikan kontribusi nyata dalam mendukung kebutuhan industri di Indonesia.

1.2. Ruang Lingkup

Penelitian ini berfokus pada pengembangan sistem deteksi kesenjangan kompetensi dan rekomendasi *skill* pada domain teknologi informasi, dengan batasan ruang lingkup sebagai berikut.

1. Domain kompetensi: penelitian ini mencakup empat subdomain utama dalam bidang IT, yaitu pengembangan perangkat lunak (*software development*), rekayasa data (*data engineering*), keamanan siber (*cybersecurity*), dan komputasi awan (*cloud computing*), yang merujuk pada kerangka kompetensi SKKNI (Standar Kompetensi Kerja Nasional Indonesia) bidang IT serta SFIA (*Skills Framework for the Information Age*).
2. Data: penelitian ini menggunakan dua sumber data utama, yaitu *dataset* JD2Skills yang diunduh dari portal lowongan kerja mycareersfuture.sg (Bhola dkk., 2020), berisi 20.298 data lowongan kerja IT yang setelah proses pembersihan menghasilkan 4.667 baris data berkualitas dengan atribut kategori pekerjaan, judul pekerjaan, deskripsi pekerjaan, IT *skills*, soft *skills*, education, experience, dan *token* usage, serta *dataset* anotasi NER (Named Entity Recognition) berisi 418.870 *token* yang digunakan untuk mendukung ekstraksi keterampilan dari teks deskripsi pekerjaan.
3. Bahasa: pemrosesan teks dilakukan dalam dua bahasa, yaitu Bahasa Indonesia dan Bahasa Inggris, mengingat praktik penulisan deskripsi pekerjaan di Indonesia yang umumnya bersifat bilingual.
4. Cakupan kompetensi: sistem yang dikembangkan dibatasi pada penilaian kompetensi teknis (*hard skills*) dan tidak mencakup evaluasi mendalam terhadap kompetensi non-teknis (*soft skills*).

5. Keluaran sistem: hasil sistem berupa identifikasi *gap* kompetensi beserta rekomendasi *skill* yang relevan, dan tidak mencakup implementasi penuh dalam lingkungan produksi organisasi.

1.3. Tujuan Penelitian

Berdasarkan rumusan masalah yang telah ditetapkan, penelitian ini memiliki empat tujuan utama yang ingin dicapai:

1. Merancang dan mengembangkan arsitektur sistem deteksi kesenjangan kompetensi pada bidang teknologi informasi yang mampu memproses data profil profesional dan deskripsi pekerjaan secara otomatis menggunakan pendekatan *natural language processing* (NLP) dan *machine learning*.
2. Mengidentifikasi dan menentukan algoritma *machine learning multi-label* yang optimal dalam memprediksi kebutuhan *skill* pada suatu kategori pekerjaan IT, dengan membandingkan beberapa metode seperti *OneVsRest Logistic Regression*, *OneVsRest SGDClassifier*, dan *Random Forest Multi-Output* berdasarkan metrik evaluasi top-k seperti *precision@k*, *recall@k*, dan *F1@k*.
3. Mengembangkan modul rekomendasi keterampilan yang dipersonalisasi berbasis prediksi *top-N skill*, untuk menghasilkan rekomendasi *skill* prioritas yang relevan berdasarkan hasil deteksi kesenjangan antara *skill* yang dimiliki pengguna dan kebutuhan *skill* pada kategori pekerjaan target.
4. Mengevaluasi kinerja sistem secara menyeluruh, baik dari aspek performa model prediksi *skill multi-label* maupun kualitas rekomendasi yang dihasilkan, sehingga dapat diketahui tingkat efektivitas sistem dalam mendeteksi kesenjangan kompetensi dan memberikan rekomendasi keterampilan pada konteks industri IT di Indonesia.

1.4. Metode Penelitian

Penelitian ini menggunakan pendekatan kuantitatif eksperimental dengan metodologi pengembangan sistem yang mengacu pada kerangka kerja CRISP-DM (*Cross-Industry Standard Process for Data Mining*), yang terdiri atas enam tahapan sistematis sebagai berikut.

1. *Business Understanding*

Pada tahap ini dilakukan identifikasi terhadap permasalahan utama yang menjadi dasar penelitian, yaitu adanya kesenjangan kompetensi (*skill gap*) antara kemampuan yang dimiliki profesional IT dengan kebutuhan kompetensi yang dibutuhkan industri. Kondisi tersebut menyebabkan individu mengalami kesulitan dalam menentukan keterampilan yang perlu ditingkatkan agar sesuai dengan tuntutan pekerjaan (Kistianingrum et al., 2026). Berdasarkan permasalahan tersebut, ditetapkan tujuan penelitian yaitu membangun sistem yang mampu mendeteksi kesenjangan kompetensi secara otomatis serta memberikan rekomendasi pengembangan keterampilan yang sesuai. Pada tahap ini juga ditentukan standar kompetensi yang digunakan sebagai acuan, yaitu SKKNI dan SFIA, sehingga hasil analisis yang dihasilkan memiliki dasar yang jelas dan relevan dengan kebutuhan industri teknologi informasi.

2. *Data Understanding*

Tahap ini bertujuan untuk memahami karakteristik dataset yang digunakan dalam penelitian. Penelitian ini menggunakan dua dataset utama, yaitu dataset JD2Skills yang diunduh dari portal lowongan kerja mycareersfuture.sg (Bhola et al., 2020) berupa berkas JSON mentah berisi 20.298 lowongan kerja IT yang menyediakan data terkait kompetensi, keterampilan profesional IT, dan kebutuhan keterampilan pada berbagai bidang pekerjaan teknologi informasi, serta dataset anotasi NER (*Named Entity Recognition*) berisi 418.870 token yang digunakan untuk mendukung ekstraksi keterampilan dari teks deskripsi pekerjaan. Kegiatan yang dilakukan meliputi pemeriksaan jumlah data, jumlah atribut, tipe data,

distribusi data, keberadaan data kosong (*missing values*), serta data duplikat. Selain itu, dilakukan analisis eksploratif untuk memahami hubungan antar variabel dan mengidentifikasi fitur-fitur yang relevan dalam proses deteksi kesenjangan kompetensi dan rekomendasi keterampilan.

3. *Data Preparation*

Pada tahap ini dilakukan proses persiapan data agar dapat digunakan dalam proses pemodelan. Kegiatan yang dilakukan meliputi pembersihan data (*data cleaning*), penanganan *missing values*, penghapusan data duplikat, normalisasi daftar *skill* agar konsisten terhadap variasi penulisan, serta penyusunan daftar kandidat *skill* yang menjadi *label* pada model. Representasi fitur dibangun menggunakan metode TF-IDF atas *skill* yang dimiliki pengguna, sedangkan kategori pekerjaan target direpresentasikan menggunakan *one-hot encoding*. Data kemudian dibagi menjadi data latih dan data uji (*train-test split*). Hasil dari tahap ini berupa *dataset* yang telah siap digunakan untuk proses pelatihan model *machine learning*.

4. *Modeling*

Pada tahap ini dilakukan pembangunan model *machine learning* untuk memprediksi kebutuhan *skill* pada suatu kategori pekerjaan IT. Permasalahan dimodelkan sebagai klasifikasi *multi-label*, di mana setiap *skill* dianggap sebagai *label* independen. *Dataset* dibagi menjadi data latih dan data uji agar proses evaluasi dapat dilakukan secara objektif. Beberapa algoritma digunakan dan dibandingkan performanya, yaitu *OneVsRest Logistic Regression*, *OneVsRest SGDClassifier*, dan *Random Forest Multi-Output*. Selain itu, dikembangkan modul rekomendasi keterampilan berbasis hasil prediksi *top-N skill*, dengan membandingkan *skill* yang dimiliki pengguna terhadap *skill* yang dibutuhkan kategori pekerjaan target sehingga dihasilkan rekomendasi *skill* yang perlu dipelajari.

5. *Evaluation*

Pada tahap ini dilakukan evaluasi terhadap performa model yang telah dibangun menggunakan data uji. Karena permasalahan bersifat *multi-label*, metrik evaluasi yang digunakan meliputi *precision@k*, *recall@k*, dan

$F1@k$ untuk mengukur kualitas prediksi top-k *skill*, serta *Hamming loss* untuk mengukur tingkat kesalahan prediksi *label* secara keseluruhan. Hasil evaluasi digunakan untuk menentukan model terbaik yang akan digunakan dalam sistem.

6. *Deployment*

Pada penelitian ini, tahap *deployment* dibatasi pada pembangunan prototipe sistem berbasis web yang mengintegrasikan modul deteksi kesenjangan kompetensi dan rekomendasi keterampilan. Sistem dikembangkan menggunakan FastAPI sebagai *backend* dan React.js sebagai *frontend*. Prototipe yang dihasilkan memungkinkan pengguna memasukkan data kompetensi yang dimiliki melalui tiga skenario, yaitu pemindaian CV, *self assessment*, dan *input* manual, untuk melihat hasil analisis kesenjangan kompetensi serta memperoleh rekomendasi keterampilan yang relevan guna meningkatkan kesesuaian dengan kebutuhan industri.

1.5 Sistematika Penulisan

Penulisan ilmiah ini disusun dengan sistematika sebagai berikut.

1. PENDAHULUAN

Berisi tulisan yang mencakup latar belakang, batasan masalah, tujuan penelitian, metode penelitian, dan sistematika penulisan.

2. LANDASAN TEORI

Membahas konsep-konsep yang menjadi dasar penelitian, meliputi kesenjangan kompetensi, standar kompetensi kerja (SKKNI dan SFIA), *natural language processing*, ekstraksi fitur teks (TF-IDF), algoritma klasifikasi, serta kerangka kerja CRISP-DM.

3. PEMBAHASAN DAN IMPLEMENTASI

Menguraikan gambaran umum penelitian, perancangan sistem, serta tahapan CRISP-DM yang meliputi *business understanding*, *data understanding*, *data preparation*, *modeling*, evaluasi, dan *deployment* sistem ElvIT.

4. PENUTUP

Menyajikan kesimpulan dari penelitian serta saran untuk pengembangan lebih lanjut.

2. Tinjauan Pustaka

2.1. Kesenjangan Kompetensi (*Skill Gap*)

Kesenjangan kompetensi atau *skill gap* merupakan kondisi ketidaksesuaian antara kemampuan yang dimiliki oleh tenaga kerja dengan kompetensi yang sesungguhnya dibutuhkan oleh suatu pekerjaan atau industri. Menurut (*Skill Gap | CEDEFOP, 2023*), *skill gap* terjadi ketika keterampilan yang dimiliki individu tidak memenuhi standar kompetensi yang dipersyaratkan oleh suatu jabatan, sehingga berdampak pada penurunan produktivitas kerja dan efektivitas organisasi secara keseluruhan. Kondisi tersebut terbukti masih berlangsung hingga saat ini; pemetaan literatur yang dilakukan oleh (Diniz et al., 2024) terhadap sejumlah studi pada industri perangkat lunak menunjukkan bahwa kesenjangan antara kompetensi lulusan pendidikan tinggi dengan kebutuhan industri tetap menjadi persoalan yang berulang, karena kebutuhan kompetensi terus bergeser seiring munculnya teknologi baru sehingga proses identifikasinya tidak dapat dilakukan secara statis melainkan harus diperbarui secara berkelanjutan.

Identifikasi *skill gap* secara konvensional umumnya dilakukan melalui wawancara, kuesioner, atau penilaian berbasis observasi langsung oleh atasan maupun asesor. Pendekatan ini memiliki keterbatasan dari segi skalabilitas, konsistensi penilaian, serta potensi bias subjektif dari penilai. Sebagai alternatif, penelitian NLP-driven yang disurvei oleh (Senger et al., 2024) menunjukkan bahwa pemanfaatan data digital seperti profil profesional dan deskripsi lowongan kerja melalui pendekatan komputasional berbasis pembelajaran mendalam semakin banyak diadopsi karena dinilai lebih objektif, terukur, dan efisien dibandingkan metode penilaian manual.

2.2. Standar Kompetensi Kerja

Standar Kompetensi Kerja adalah rumusan mengenai kemampuan yang wajib dimiliki seseorang agar dapat menyelesaikan tugas atau pekerjaan secara

tepat, mencakup tiga aspek utama: pengetahuan, keterampilan, dan sikap kerja yang relevan dengan pelaksanaan tugas serta syarat jabatan tertentu. Secara sederhana, ini adalah tolok ukur atau pedoman yang disepakati bersama untuk menilai apakah seseorang cukup mampu menjalankan suatu pekerjaan. Penelitian ini menggunakan dua kerangka standar kompetensi, yaitu SKKNI sebagai acuan nasional dan SFIA sebagai acuan internasional, yang diuraikan pada dua subbagian berikut.

2.2.1. SKKNI (Standar Kompetensi Kerja Nasional Indonesia)

SKKNI merupakan rumusan kemampuan kerja yang mencakup aspek pengetahuan, keterampilan, dan sikap kerja yang relevan dengan pelaksanaan tugas serta syarat jabatan yang ditetapkan sesuai dengan ketentuan perundang-undangan yang berlaku di Indonesia. Menurut (*Keputusan Menteri Ketenagakerjaan Nomor 30 Tahun 2025, 2025*), SKKNI disusun untuk menjadi acuan dalam penyusunan program pendidikan dan pelatihan vokasi, uji kompetensi, serta pengembangan jenjang karier tenaga kerja di berbagai sektor, termasuk sektor teknologi informasi. Penggunaan SKKNI sebagai acuan penyusunan materi dan program pelatihan berbasis kompetensi juga dikonfirmasi oleh kajian (Sobirin, 2020), yang menegaskan bahwa pendidikan berbasis SKKNI membekali lulusan dengan kemampuan kerja mencakup pengetahuan, keterampilan, dan sikap kerja yang relevan dengan syarat jabatan sesuai ketentuan yang berlaku. SKKNI bidang IT memuat unit-unit kompetensi yang terbagi ke dalam berbagai subbidang seperti pengembangan perangkat lunak, jaringan, keamanan informasi, dan manajemen data.

2.2.2. SFIA (*Skills Framework for the Information Age*)

SFIA adalah kerangka kerja kompetensi internasional yang digunakan secara luas untuk mendeskripsikan keterampilan dan kompetensi profesional di bidang teknologi informasi dan digital. Menurut (*How SFIA Works - Levels of Responsibility and Skills — English, n.d.*), kerangka ini mengelompokkan kompetensi ke dalam beberapa kategori area kerja dan tingkat kemahiran (levels of

responsibility), mulai dari level dasar hingga level strategis, sehingga dapat digunakan untuk memetakan jenjang karier maupun kebutuhan pelatihan secara terstruktur. Relevansi praktis kerangka ini turut ditunjukkan oleh (Bowers & Sabin, 2022), yang menerapkan SFIA untuk menilai lebih dari 120 keterampilan profesional IT di tujuh tingkat tanggung jawab guna mendukung asesmen kompetensi lulusan pendidikan tinggi terhadap kebutuhan dunia kerja. Penggunaan SFIA bersama SKKNI dalam penelitian ini bertujuan untuk menghasilkan pemetaan kompetensi yang relevan baik dengan standar nasional maupun praktik industri global.

2.3. *Natural Language Processing (NLP)*

Natural Language Processing (NLP) adalah cabang ilmu kecerdasan buatan yang berfokus pada kemampuan komputer untuk memahami, mengolah, dan menghasilkan bahasa manusia baik dalam bentuk teks maupun ucapan. Menurut (Liddy, 2001), NLP merupakan serangkaian teknik komputasi yang dilandasi teori linguistik, digunakan untuk menganalisis dan merepresentasikan teks yang terjadi secara alami pada satu atau beberapa tingkat analisis linguistik, dengan tujuan mencapai pemrosesan bahasa yang menyerupai cara manusia memahami bahasa untuk berbagai jenis tugas atau aplikasi. Perkembangan NLP turut mendorong munculnya bidang analisis pasar kerja berbasis komputasi; tinjauan yang dilakukan (Senger et al., 2024) menegaskan bahwa kemajuan NLP dalam beberapa tahun terakhir mempercepat penerapan metode ekstraksi dan klasifikasi kompetensi dari teks lowongan kerja maupun profil kandidat. Dalam konteks penelitian ini, NLP digunakan untuk mengekstraksi informasi kompetensi yang terkandung dalam teks profil profesional dan deskripsi pekerjaan yang bersifat tidak terstruktur.

2.3.1. *Text Preprocessing*

Text preprocessing merupakan tahapan awal dalam pengolahan data teks yang bertujuan membersihkan dan menyeragamkan format teks sebelum dilakukan proses ekstraksi fitur. Pada sistem ElvIT, tahapan ini meliputi *cleaning* teks dengan

menghapus karakter non-alfanumerik, penyeragaman huruf (*case folding*), serta tokenisasi teks berdasarkan pemisah umum seperti koma, titik koma, dan baris baru. Selanjutnya, daftar *skill* pengguna dinormalisasi dan dicocokkan terhadap daftar kandidat *skill* yang telah disusun dari *dataset* untuk menjaga konsistensi representasi. Pendekatan pembersihan dan normalisasi berbasis aturan ini dipilih karena data *skill* yang digunakan bersifat terstruktur, sehingga tidak memerlukan *stemming* maupun penghapusan *stopword*.

2.3.2. Ekstraksi Teks PDF dan *Named Entity Recognition*

Ekstraksi teks dokumen PDF dilakukan menggunakan pustaka PyMuPDF yang membaca seluruh teks pada *Curriculum Vitae* guna diproses lebih lanjut. Setelah teks diperoleh, *skill* pengguna dideteksi menggunakan pencocokan eksak dan *substring* terhadap daftar *skill* yang diketahui. Sebagai lapisan *fallback* tambahan digunakan model bahasa alami spaCy (*en_core_web_sm*) untuk mengekstraksi *noun-chunk* yang berpotensi menjadi keterampilan, serta *dataset* anotasi NER untuk menyusun kosakata keterampilan yang digunakan dalam sistem. Pendekatan serupa diterapkan oleh (Sinha et al., 2023), yang mengombinasikan PyMuPDF untuk konversi dokumen PDF ke teks dengan spaCy untuk *named entity recognition* guna mengekstraksi entitas keterampilan dari dokumen *resume*, dan melaporkan bahwa kombinasi tersebut mampu meningkatkan akurasi prediksi domain pekerjaan hingga di atas 92%.

2.4. Ekstraksi Fitur Teks

Ekstraksi fitur dilakukan untuk mengubah data teks menjadi representasi numerik yang dapat diproses oleh algoritma *machine learning*. Pada penelitian ini digunakan dua jenis representasi, yaitu TF-IDF untuk merepresentasikan daftar *skill* yang dimiliki pengguna dan *one-hot encoding* untuk merepresentasikan kategori pekerjaan target.

2.4.1. TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF merupakan metode pembobotan kata yang digunakan untuk mengukur tingkat kepentingan suatu kata dalam sebuah dokumen relatif terhadap kumpulan dokumen secara keseluruhan (corpus) (Yunitarini et al., 2025), metode ini bekerja dengan menggabungkan frekuensi kemunculan kata pada suatu dokumen (term frequency) dengan tingkat keunikan kata tersebut pada seluruh dokumen (inverse document frequency), sehingga kata yang sering muncul pada satu dokumen namun jarang muncul pada dokumen lain akan memiliki bobot yang lebih tinggi. Kajian literatur oleh (Setiawan et al., 2022) turut menegaskan bahwa TF-IDF masih banyak digunakan sebagai representasi fitur awal sebelum data teks diproses lebih lanjut menggunakan algoritma *machine learning*, khususnya pada tugas klasifikasi teks berskala besar.

2.4.2. One-Hot Encoding

One-hot encoding adalah teknik representasi kategorikal yang mengubah nilai kategori menjadi vektor biner dengan panjang sejumlah kategori yang ada. Setiap kategori direpresentasikan oleh vektor dengan nilai 1 pada indeks kategori tersebut dan 0 pada indeks lainnya. Studi komparatif oleh (Zhu et al., 2024) yang mengevaluasi 14 metode *encoding* kategorikal pada 28 *dataset* membuktikan secara teoretis maupun empiris bahwa *one-hot encoding* merupakan pilihan paling sesuai untuk model yang melakukan transformasi affine pada *input*, seperti regresi logistik, karena mampu merepresentasikan kategori tanpa kehilangan informasi. Dalam sistem ElvIT, kategori pekerjaan dikodekan menggunakan *one-hot encoding* sehingga informasi target pekerjaan dapat dikombinasikan dengan fitur TF-IDF sebagai *input* model.

2.5. Klasifikasi *Multi-Label*

Permasalahan yang diselesaikan pada penelitian ini bersifat *multi-label*, yaitu memprediksi lebih dari satu *skill* yang dibutuhkan untuk suatu kategori pekerjaan. Sifat *multi-label* pada tugas prediksi kompetensi ini sejalan dengan

temuan (Senger et al., 2024), yang mencatat bahwa mayoritas penelitian ekstraksi dan klasifikasi *skill* dari lowongan kerja memodelkan tugas tersebut sebagai persoalan *multi-label* karena satu dokumen dapat memuat lebih dari satu kompetensi sekaligus. Pendekatan yang digunakan pada penelitian ini adalah *binary relevance* dengan strategi OneVsRest, di mana setiap kandidat *skill* diperlakukan sebagai *label* biner dan dilatih secara independen oleh sebuah model dasar. Beberapa model yang dibandingkan adalah OneVsRest *Logistic Regression*, OneVsRest *SGDClassifier*, dan *Random Forest (Multi-Output)*.

2.5.1. *Binary Relevance* dan *OneVsRest Classifier*

Binary relevance merupakan strategi klasifikasi *multi-label* yang mengubah permasalahan *multi-label* menjadi sejumlah permasalahan klasifikasi biner, satu untuk setiap *label*. Kelas *OneVsRestClassifier* menerapkan strategi ini dengan melatih satu model dasar untuk setiap *label*. Eksperimen (Arslan & Cruz, 2023) yang membandingkan *binary relevance*, *classifier chain*, dan *label powerset* pada tugas klasifikasi teks *multi-label* menemukan bahwa strategi *binary relevance* memberikan performa yang jauh lebih baik dibandingkan *classifier chain* maupun *label powerset* pada *dataset* dengan ruang *label* yang cukup besar. Pada sistem ElvIT, setiap kandidat *skill* menjadi sebuah *label* biner yang memprediksi apakah *skill* tersebut dibutuhkan untuk kategori pekerjaan tertentu, sehingga model menghasilkan probabilitas kebutuhan seluruh kandidat *skill* dalam satu proses inferensi.

2.5.2. *Logistic Regression*

Logistic Regression adalah algoritma klasifikasi linier yang memodelkan probabilitas suatu sampel masuk ke dalam kelas tertentu menggunakan fungsi logistik (sigmoid). Algoritma ini bekerja dengan mencari bobot optimal untuk setiap fitur melalui proses optimasi seperti *gradient descent*. Karakteristik tersebut membuat *Logistic Regression* tetap menjadi salah satu baseline yang paling umum digunakan pada studi klasifikasi biner maupun *multi-label*; perbandingan sejumlah

algoritma klasifikasi yang dilakukan (Putra & Rumini, 2025) menunjukkan bahwa *Logistic Regression* tetap kompetitif sebagai model dasar pada data yang tidak seimbang meskipun umumnya memberikan performa yang sedikit lebih rendah dibandingkan model ensemble seperti *Random Forest*. Dalam penelitian ini, *Logistic Regression* digunakan sebagai model dasar pada strategi OneVsRest untuk menghasilkan probabilitas kebutuhan setiap kandidat *skill*.

2.5.3. *Stochastic Gradient Descent Classifier (SGDClassifier)*

Stochastic Gradient Descent (SGD) Classifier merupakan algoritma klasifikasi linier yang melatih model menggunakan optimasi *gradient descent* berbasis sampel acak (stochastic). *SGDClassifier* efisien untuk data berdimensi besar dan mendukung fungsi *loss* logistik sehingga dapat menghasilkan probabilitas kelas. *SGDClassifier* dirancang untuk data yang jarang (sparse) dan berdimensi tinggi seperti pada klasifikasi teks, serta mampu diskalakan pada permasalahan dengan lebih dari 100.000 fitur (*1.5. Stochastic Gradient Descent — Scikit-Learn 1.9.0 Documentation*, n.d.). Pada pengujian benchmark klasifikasi dokumen teks, model-model linier seperti *SGDClassifier*, *Ridge Classifier*, dan *Linear SVC* terbukti lebih sesuai untuk data berdimensi tinggi dibandingkan model berbasis pohon keputusan seperti *Random Forest*, yang justru lambat dilatih, mahal saat prediksi, dan memiliki akurasi yang relatif lebih rendah pada data semacam ini (*Classification of Text Documents Using Sparse Features — Scikit-Learn 1.9.0 Documentation*, n.d.). Pada penelitian ini, *SGDClassifier* digunakan sebagai model dasar alternatif dalam strategi OneVsRest untuk membandingkan performanya dengan *Logistic Regression*.

2.5.4. *Random Forest*

Random Forest merupakan algoritma *ensemble learning* yang membangun banyak pohon keputusan (*decision tree*) secara acak dan menggabungkan hasil prediksi dari masing-masing pohon untuk menghasilkan keputusan akhir melalui mekanisme voting (untuk klasifikasi) atau rata-rata (untuk regresi). Menurut

(Breiman, 2001), pendekatan ensemble ini terbukti mampu mengurangi risiko overfitting yang umum terjadi pada pohon keputusan tunggal, sekaligus meningkatkan akurasi serta stabilitas model terhadap data baru. Temuan tersebut masih relevan hingga saat ini; analisis komparatif oleh (Fatima et al., 2023) terhadap *Random Forest* dan algoritma XGBoost sebagai representasi metode ensemble lain melaporkan bahwa *Random Forest* secara konsisten unggul dibandingkan model linier tunggal pada mayoritas *dataset* yang diuji, terutama pada data dengan interaksi fitur yang kompleks. Pada penelitian ini, *Random Forest* digunakan dalam mode *multi-output* yang mampu memprediksi seluruh *label skill* secara bersamaan.

2.6. Sistem Rekomendasi *Skill*

Sistem rekomendasi pada ElvIT berbasis hasil prediksi model *multi-label*. Model memprediksi *top-N skill* yang dibutuhkan untuk kategori pekerjaan target berdasarkan *skill* yang dimiliki pengguna. *Skill* prediksi yang tidak terdapat pada daftar *skill* pengguna menjadi *gap* yang kemudian diprioritaskan sebagai rekomendasi belajar. Pendekatan berbasis prediksi semacam ini sejalan dengan arah perkembangan riset ekstraksi dan klasifikasi *skill* yang dirangkum (Senger et al., 2024), yang menunjukkan bahwa hasil klasifikasi kompetensi dari suatu model semakin banyak dimanfaatkan sebagai dasar sistem pencocokan dan rekomendasi pekerjaan yang bersifat personal. Dengan demikian, rekomendasi bersifat personal karena dibentuk berdasarkan daftar *skill* pengguna dan kategori pekerjaan yang dipilih.

Rekomendasi dibentuk dengan mengambil *N skill* dengan probabilitas tertinggi dari *output* model (*top-N skill*). Setiap *skill* dibandingkan dengan *skill* pengguna menggunakan pencocokan *substring*; *skill* yang sudah dimiliki masuk ke dalam kategori *matched*, sedangkan sisanya merupakan *gap skill*. Seluruh *gap skill* diurutkan berdasarkan probabilitas prediksi dan diberi *priority rank*, sehingga rekomendasi belajar yang paling mendesak ditampilkan lebih dahulu.

2.7. CRISP-DM (Cross-Industry Standard Process for Data Mining)

CRISP-DM merupakan kerangka kerja standar yang digunakan dalam proses pengembangan proyek data mining dan *machine learning*, yang terdiri atas enam tahapan utama, yaitu *business understanding*, *data understanding*, *data preparation*, *modeling*, *evaluation*, dan *deployment*. Menurut (Chapman et al., 2000), kerangka kerja ini bersifat iteratif dan fleksibel, sehingga memungkinkan peneliti untuk kembali ke tahapan sebelumnya apabila ditemukan permasalahan pada tahap selanjutnya, dengan tujuan menghasilkan solusi data mining yang sesuai dengan kebutuhan bisnis maupun penelitian. Relevansi kerangka kerja ini masih terjaga hingga saat ini; tinjauan sistematis oleh (Schröer et al., 2021) terhadap literatur penerapan CRISP-DM menyimpulkan bahwa kerangka ini tetap menjadi model proses yang paling banyak diadopsi pada proyek data mining maupun *machine learning* lintas industri karena sifatnya yang generik dan mudah diadaptasi ke berbagai jenis proyek.

2.8. Evaluasi Model

Evaluasi model dilakukan untuk mengetahui sejauh mana model mampu menghasilkan prediksi yang benar. Pada permasalahan klasifikasi *multi-label* dengan rekomendasi *top-N*, evaluasi tidak cukup hanya menggunakan *accuracy*, *precision*, *recall*, dan *F1-Score* pada *threshold* tunggal, tetapi juga menggunakan metrik berbasis peringkat (*ranking*) seperti *precision@k*, *recall@k*, dan *F1@k*, serta metrik tambahan seperti *Hamming loss*, *confusion matrix multi-label*, dan kurva ROC serta *Precision-Recall*.

Akurasi, *precision*, *recall*, dan *F1-Score* tetap digunakan untuk menilai kualitas klasifikasi pada *threshold* tertentu. Namun, karena sistem ElvIT menampilkan rekomendasi *skill* dalam bentuk daftar *top-N*, *F1@10* dipilih sebagai metrik utama pemilihan model karena merepresentasikan keseimbangan antara ketepatan dan cakupan *skill* relevan pada sepuluh rekomendasi teratas yang ditampilkan kepada pengguna.

2.8.1. Metrik Top-K (*Precision@k*, *Recall@k*, *F1@k*)

Metrik top-k mengevaluasi kualitas rekomendasi berdasarkan k *skill* teratas dengan probabilitas tertinggi. *Precision@k* mengukur proporsi *skill* relevan pada k rekomendasi teratas, *recall@k* mengukur proporsi *skill* relevan yang tertangkap dalam k rekomendasi tersebut, dan *F1@k* merupakan rata-rata harmonik keduanya. Penggunaan ketiga metrik ini untuk mengevaluasi kualitas rekomendasi top-k juga diterapkan oleh (He et al., 2024) pada tugas rekomendasi tag, yang menunjukkan bahwa kombinasi *precision@k*, *recall@k*, dan *F1-score@k* mampu menangkap *trade-off* antara ketepatan dan cakupan rekomendasi secara lebih representatif dibandingkan metrik akurasi tunggal. Dalam sistem ElvIT, *F1@10* digunakan sebagai metrik utama pemilihan model karena aplikasi menampilkan rekomendasi *skill* dalam bentuk daftar *top-N*.

2.8.2. Hamming Loss

Hamming loss mengukur proporsi kesalahan klasifikasi pada level *label*, yaitu perbandingan jumlah *label* yang diprediksi salah terhadap total seluruh pasangan *instance-label*. Nilai yang semakin mendekati nol menunjukkan tingkat kesalahan klasifikasi yang semakin kecil. Penerapan *Hamming loss* sebagai metrik pelengkap pada evaluasi klasifikasi *multi-label* ditunjukkan oleh (Lentzas et al., 2022), yang menggunakan metrik ini bersama *F1-Score* untuk menilai kesalahan klasifikasi pada level *label* secara lebih rinci dibandingkan metrik berbasis subset *accuracy* yang cenderung terlalu ketat.

2.8.3. Confusion Matrix Multi-Label

Confusion matrix digunakan untuk melihat pola prediksi benar dan salah pada level *label*. Pada konteks *multi-label*, matriks disusun per *label skill* sehingga dapat diketahui seberapa sering sebuah *skill* berhasil direkomendasikan saat memang dibutuhkan, serta seberapa sering *skill* tidak terdeteksi meskipun sebenarnya dibutuhkan.

2.8.4. Kurva ROC dan *Precision-Recall*

Kurva ROC (*Receiver Operating Characteristic*) membandingkan *true positive rate* dengan *false positive rate*, sedangkan kurva *Precision-Recall* membandingkan *precision* dengan *recall* pada berbagai *threshold*. Keduanya menggambarkan kemampuan model dalam membedakan *label skill* dan kualitas peringkat rekomendasi yang dihasilkan. Tinjauan metrik evaluasi sistem rekomendasi oleh (Deldjoo et al., 2024) menegaskan bahwa kurva ROC dan *Precision-Recall* tetap menjadi alat visualisasi standar untuk menilai *trade-off* antar *threshold* pada model klasifikasi maupun rekomendasi berbasis peringkat, sehingga keduanya digunakan sebagai pelengkap metrik top-k pada penelitian ini.

2.9. Penelitian Terdahulu

Beberapa penelitian terdahulu yang relevan dengan topik deteksi kesenjangan kompetensi (*skill gap*) berbasis *machine learning* dan NLP dirangkum pada Tabel 2.1 berikut

Tabel 2.1. Penelitian Terdahulu

No	Peneliti (Tahun)	Judul Penelitian	Metode	Hasil
1	Sinha et al. (2023)	<i>Automated resume parsing and job domain prediction using machine learning</i>	Ekstraksi teks CV menggunakan PyMuPDF dikombinasikan dengan <i>Named Entity Recognition</i> (spaCy) untuk mendeteksi entitas <i>skill</i>	Kombinasi PyMuPDF dan spaCy meningkatkan akurasi prediksi domain pekerjaan hingga di atas 92%
2	Diniz et al. (2024)	<i>The skill gap in software industry: A mapping study</i>	Studi pemetaan literatur (<i>mapping study</i>) terhadap penelitian <i>skill gap</i> pada industri perangkat lunak	Kesenjangan kompetensi lulusan pendidikan tinggi dengan kebutuhan industri tetap menjadi persoalan berulang seiring pergeseran kebutuhan teknologi
3	Senger et al. (2024)	<i>Deep learning-based computational job market analysis: A survey on skill extraction and classification from job postings</i>	Survei metode NLP dan <i>deep learning</i> untuk ekstraksi serta klasifikasi <i>skill</i> dari teks lowongan kerja	Mayoritas penelitian memodelkan ekstraksi <i>skill</i> sebagai persoalan klasifikasi <i>multi-label</i> karena satu dokumen dapat memuat lebih dari satu kompetensi
4	Goyal et al. (2023)	<i>JobXMLC: EXtreme multi-label classification of job skills with graph neural networks</i>	<i>Graph neural network</i> (GNN) dengan representasi <i>job-skill graph</i> dan mekanisme <i>skill attention</i> untuk <i>extreme multi-label classification</i>	JobXMLC meningkatkan <i>precision</i> dan <i>recall</i> secara signifikan dibandingkan pendekatan <i>extreme multi-label classification</i> sebelumnya pada <i>dataset</i> rekrutmen berskala besar
5	Leon et al. (2024)	<i>Hierarchical classification of transversal skills in job advertisements based on sentence embeddings</i>	Klasifikasi hierarkis <i>multi-label</i> berbasis <i>sentence embeddings</i> (Sentence-BERT) dengan taksonomi ESCO, dilengkapi augmentasi data untuk mengatasi ketidakseimbangan kelas	Model <i>sentence embedding</i> <i>multi-language</i> mencapai akurasi yang sebanding dengan model khusus Bahasa Inggris dalam klasifikasi <i>skill</i> transversal dari iklan lowongan kerja

Berdasarkan penelitian-penelitian terdahulu tersebut, terlihat bahwa pendekatan berbasis TF-IDF, ekstraksi teks CV dengan *Named Entity Recognition*, *graph neural network*, maupun *sentence embeddings* telah banyak digunakan dan terbukti efektif untuk klasifikasi *skill*, termasuk dalam skema *multi-label* seperti pada JobXMLC (Goyal et al., 2023) dan klasifikasi hierarkis *skill* transversal (Leon et al., 2024). Namun demikian, penelitian-penelitian tersebut umumnya berfokus pada ekstraksi maupun klasifikasi *skill* dari teks lowongan kerja secara generik dan belum mengacu pada kerangka standar kompetensi formal seperti SKKNI maupun SFIA, serta belum secara khusus mengevaluasi hasil klasifikasi menggunakan metrik berbasis peringkat (*precision@k*, *recall@k*, *F1@k*) yang relevan untuk sistem rekomendasi *top-N skill*. Penelitian ini melengkapi kesenjangan tersebut dengan memodelkan prediksi *skill* sebagai persoalan klasifikasi *multi-label* menggunakan strategi *binary relevance OneVsRest*, memanfaatkan kombinasi fitur TF-IDF dan *one-hot encoding*, serta menghasilkan rekomendasi *top-N skill* yang dievaluasi menggunakan metrik berbasis peringkat seperti *precision@k*, *recall@k*, dan *F1@k*, dengan kompetensi yang dipetakan mengacu pada standar SKKNI dan SFIA sekaligus.

3. PEMBAHASAN DAN IMPLEMENTASI

3.1 Gambaran Umum Penelitian

Penelitian ini menghasilkan sistem berbasis *web* bernama ElvIT yang digunakan untuk mendeteksi kesenjangan kompetensi dan memberikan rekomendasi *skill* pada bidang teknologi informasi. Sistem dirancang untuk menerima daftar *skill* pengguna melalui tiga jalur input, yaitu pemindaian *CV* berformat PDF, self assessment, dan input manual. Setelah *skill* pengguna diperoleh, sistem meminta pengguna memilih target kategori pekerjaan. Model machine learning kemudian memprediksi daftar *skill* yang relevan untuk kategori pekerjaan tersebut, lalu sistem membandingkannya dengan *skill* pengguna untuk menghasilkan *matched skills*, *gap skills*, *gap score*, *match score*, dan rekomendasi *skill* prioritas.

Pendekatan penelitian disusun menggunakan *CRISP-DM* karena project ini berfokus pada pemanfaatan data lowongan kerja dan pemodelan machine learning. *CRISP-DM* digunakan sebagai kerangka kerja mulai dari pemahaman masalah, pemahaman data, persiapan data, pemodelan, evaluasi, hingga deployment prototype aplikasi web. Tahapan penelitian berdasarkan *CRISP-DM* ditampilkan pada Tabel 3.1.

Tabel 3.1 Tahapan Penelitian Berdasarkan CRISP-DM

Tahap	Kegiatan	Keluaran
Business Understanding	Mengidentifikasi kebutuhan sistem deteksi gap skill dan rekomendasi skill untuk pengguna yang memiliki target pekerjaan IT tertentu.	Tujuan sistem, batasan penelitian, dan keluaran analisis yang dibutuhkan.
Data Understanding	Memahami struktur dataset lowongan kerja IT, kategori pekerjaan, kolom skill, dan karakteristik data yang tersedia.	Ringkasan dataset, atribut penting, dan gambaran distribusi data.

Tahap	Kegiatan	Keluaran
Data Preparation	Membersihkan data, menormalisasi skill, membentuk kandidat label skill, membuat fitur TF-IDF, dan melakukan encoding kategori pekerjaan.	Dataset siap latih dengan fitur numerik dan label multi-label.
Modeling	Membangun dan membandingkan beberapa model multi-label skill prediction.	Hasil evaluasi ketiga model dan komponen model final.
Evaluation	Mengevaluasi performa ketiga model menggunakan metrik multi-label dan top-k recommendation, kemudian memilih model terbaik.	Nilai Precision@K, Recall@K, F1@K, Hamming Loss, classification report, confusion matrix, dan ROC-PR curve.
Deployment	Mengintegrasikan model terbaik ke backend FastAPI dan frontend React, serta menguji alur sistem.	Prototype web ElvIT dengan fitur Scan CV, Self Assessment, Manual Skill Check, deteksi gap, dan rekomendasi skill.

Berdasarkan Tabel 3.1, tahapan penelitian dimulai dari penentuan masalah dan kebutuhan sistem, kemudian dilanjutkan dengan proses pemahaman dan pengolahan data. Setelah data siap digunakan, model *multi-label* dibangun untuk memprediksi *skill* yang dibutuhkan pada suatu kategori pekerjaan. Model yang telah dievaluasi kemudian diintegrasikan ke dalam prototype aplikasi *web ElvIT*.

3.2 Business Understanding

Tahap Business Understanding bertujuan untuk memahami permasalahan utama yang ingin diselesaikan oleh sistem. Permasalahan yang diangkat dalam penelitian ini adalah adanya kesenjangan antara *skill* yang dimiliki pengguna dan *skill* yang dibutuhkan pada pekerjaan IT tertentu. Pengguna sering kali mengetahui *skill* yang dimiliki, tetapi belum memiliki gambaran sistematis mengenai *skill* yang perlu ditingkatkan untuk mencapai target pekerjaan tertentu.

Tujuan teknis pada tahap ini adalah membangun sistem yang dapat menerima *skill* pengguna, memprediksi *skill* yang relevan untuk target pekerjaan, membandingkan keduanya, dan menghasilkan rekomendasi *skill*. Indikator

keberhasilan penelitian dilihat dari keberhasilan pipeline data, performa model *multi-label*, serta keberhasilan integrasi model ke prototype aplikasi web. Ringkasan tujuan dan keluaran sistem ditampilkan pada Tabel 3.2.

Tabel 3.2 Tujuan dan Keluaran Sistem ElvIT

No	Tujuan	Keluaran
1	Menerima input skill pengguna dari beberapa jalur.	Daftar skill hasil Scan CV, Self Assessment, atau input manual.
2	Memprediksi skill yang relevan untuk target pekerjaan.	Daftar top-N required skills beserta probabilitas model.
3	Mendeteksi kesenjangan kompetensi.	Matched skills, gap skills, gap score, dan match score.
4	Memberikan rekomendasi skill prioritas.	Daftar rekomendasi skill yang diurutkan berdasarkan prioritas model.
5	Mengintegrasikan model ke aplikasi web.	Backend FastAPI dan frontend React yang dapat menjalankan alur analisis.

Berdasarkan Tabel 3.2, keluaran utama sistem ElvIT berupa daftar *skill* yang dibutuhkan, *skill* yang sudah cocok, *skill* yang belum dimiliki, dan rekomendasi *skill*. Dengan demikian, pembahasan pada Bab 3 diarahkan pada deteksi kesenjangan kompetensi dan rekomendasi *skill*.

3.3 Data Understanding

Tahap Data Understanding dilakukan untuk memahami struktur dan karakteristik dataset yang digunakan. Dataset utama yang digunakan adalah dataset JD2Skills yang diunduh dari portal lowongan kerja mycareersfuture.sg (Bhola et al., 2020) dalam bentuk berkas JSON mentah berisi 20.298 lowongan, kemudian diproses menjadi berkas JD2Skills_processed.csv. Dataset hasil pemrosesan berisi informasi kategori pekerjaan, judul pekerjaan, deskripsi pekerjaan, IT skills, soft skills, education, experience, dan kolom pendukung lainnya. Selain itu, penelitian ini juga menggunakan dataset anotasi NER (*Named Entity Recognition*) berisi

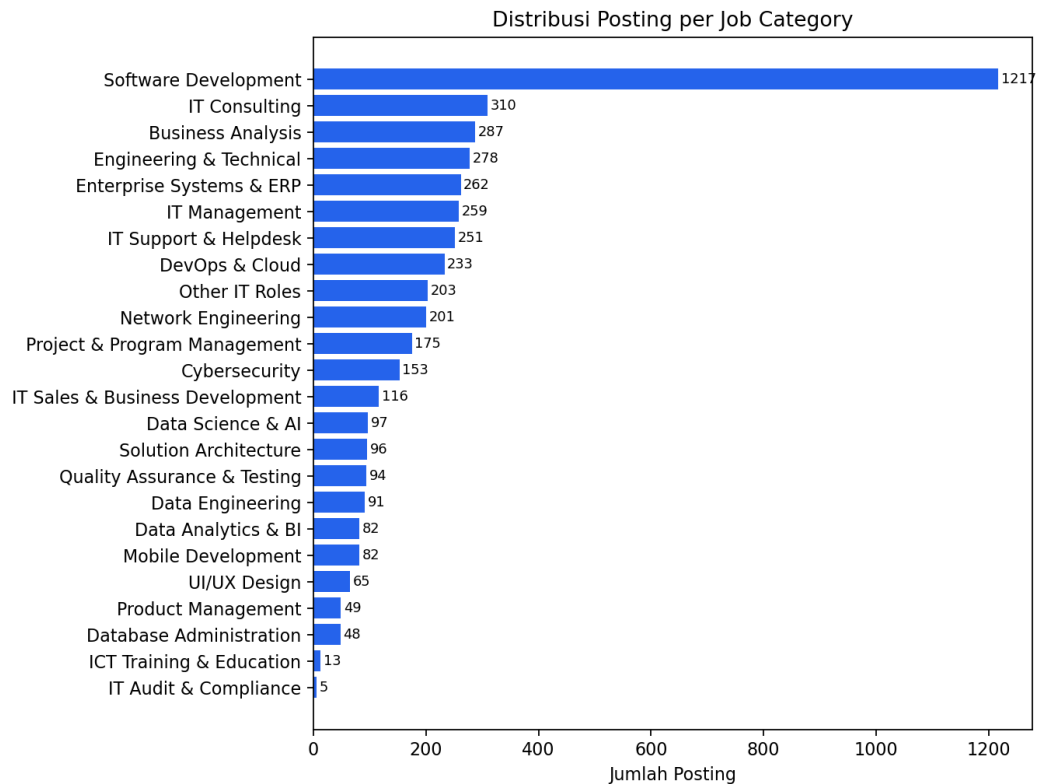
418.870 token untuk mendukung ekstraksi keterampilan dari teks deskripsi pekerjaan. Ringkasan struktur dataset ditampilkan pada Tabel 3.3.

Tabel 3.3 Ringkasan Dataset Lowongan Kerja IT

No	Komponen	Nilai
1	Jumlah data mentah	20.298 lowongan (JSON)
2	Jumlah data hasil pembersihan	4.667 baris
3	Jumlah kolom	11 kolom
4	Jumlah kategori pekerjaan	24 kategori
5	Kolom utama	Query, Job Title, IT Skills, Description, Education, Experience
6	Dataset anotasi NER	418.870 baris anotasi token

Berdasarkan Tabel 3.3, *dataset* memiliki jumlah data yang cukup untuk membentuk kandidat *skill* dan kategori pekerjaan. Namun, kolom *Soft Skills* dan *Education* memiliki nilai kosong pada seluruh baris, sedangkan kolom *Seniority* kosong pada 18 baris, sehingga project ini tidak menjadikan ketiga kolom tersebut sebagai inti prediksi akhir. Fokus utama data berada pada kategori pekerjaan (*Query*), daftar *IT Skills*, dan deskripsi pekerjaan.

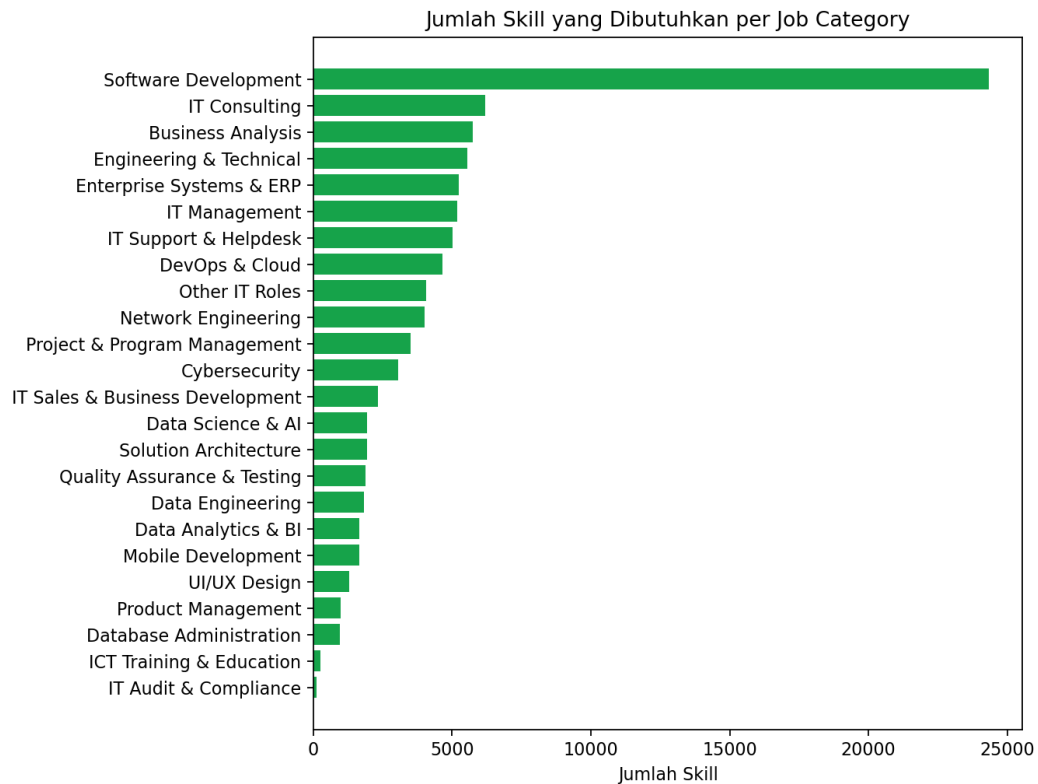
Distribusi jumlah posting pada setiap kategori pekerjaan perlu diamati untuk mengetahui penyebaran data. Visualisasi distribusi posting per kategori pekerjaan ditampilkan pada Gambar 3.1.



Gambar 3.1 Distribusi Posting per Job Category

Berdasarkan Gambar 3.1, *dataset* memuat 24 kategori pekerjaan IT dengan sebaran yang tidak merata; kategori Software Development memiliki jumlah posting terbanyak, sementara beberapa kategori seperti IT Sales & Business Development dan ICT Training & Education memiliki jumlah posting yang lebih kecil. Distribusi ini digunakan untuk memahami cakupan kategori yang dapat dipilih pengguna pada aplikasi ElvIT. Kategori pekerjaan tersebut kemudian digunakan sebagai konteks input model dan sebagai pilihan target pekerjaan pada frontend.

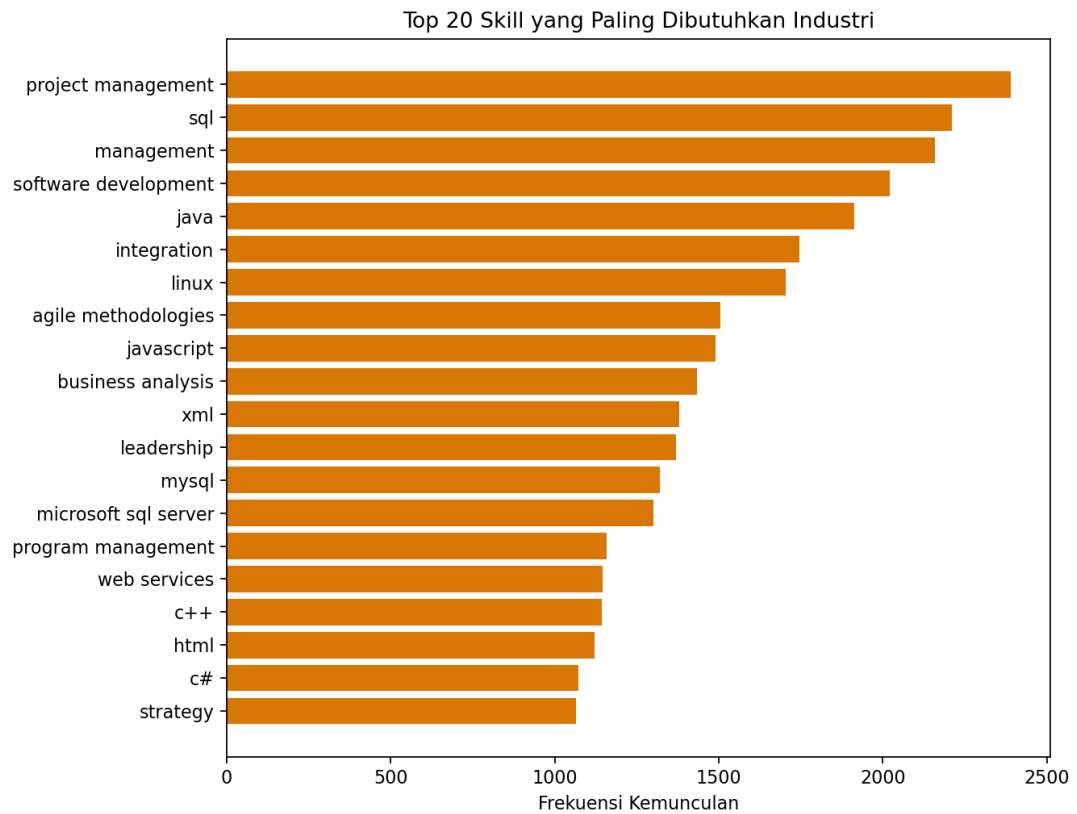
Selain distribusi kategori pekerjaan, jumlah *skill* yang dibutuhkan pada setiap kategori juga dianalisis. Distribusi jumlah *skill* yang dibutuhkan per job category ditampilkan pada Gambar 3.2.



Gambar 3.2 Distribusi Jumlah Skill yang Dibutuhkan per Job Category

Berdasarkan Gambar 3.2, setiap kategori pekerjaan memiliki kebutuhan *skill* yang berbeda, dengan rata-rata kebutuhan *skill* yang bervariasi antar kategori. Informasi ini penting karena sistem ElvIT harus menghasilkan rekomendasi yang spesifik terhadap target pekerjaan, bukan rekomendasi *skill* umum yang sama untuk semua kategori.

Skill yang paling sering muncul pada *dataset* dianalisis untuk mengetahui kebutuhan kompetensi dominan di industri IT. Daftar top *skill* yang dibutuhkan industri ditampilkan pada Gambar 3.3.



Gambar 3.3 Top 20 Skills yang Paling Dibutuhkan Industri

Berdasarkan Gambar 3.3, beberapa *skill* teknis muncul lebih dominan dibandingkan *skill* lainnya, antara lain project management (2.390 kemunculan), sql (2.211), management (2.158), software development (2.020), dan java (1.913). *Skill* yang sering muncul menjadi kandidat penting dalam proses pembentukan label *multi-label* dan menjadi bagian dari *benchmark* kebutuhan *skill* pada sistem rekomendasi.

3.4 Data Preparation

Tahap Data Preparation dilakukan untuk mengubah data mentah menjadi format yang dapat digunakan oleh model machine learning. Proses ini terdiri atas enam tahapan, yaitu pembersihan data, normalisasi *skill*, pembentukan kandidat *skill*, pembentukan fitur *TF-IDF*, *encoding* kategori pekerjaan, dan train-test split. Ringkasan keenam tahapan ditampilkan pada Tabel 3.4, kemudian setiap tahapan dijabarkan secara rinci beserta contoh hasilnya pada subbab berikut.

Tabel 3.4 Tahapan Data Preparation

No	Tahapan	Penjelasan
1	Pembersihan data	Menghapus data duplikat, menangani nilai kosong, dan menyiapkan kolom yang digunakan.
2	Normalisasi skill	Mengubah skill menjadi lowercase, menghapus spasi berlebih, dan memisahkan daftar skill.
3	Pembentukan kandidat skill	Mengambil 250 kandidat skill yang digunakan sebagai label prediksi multi-label.
4	Pembentukan fitur TF-IDF	Mengubah konteks skill pengguna menjadi representasi numerik berbasis TF-IDF.
5	Encoding kategori pekerjaan	Mengubah 24 job category menjadi fitur one-hot agar model memahami target pekerjaan.
6	Train-test split	Membagi data menjadi data latih dan data uji untuk evaluasi model.

3.4.1. Pembersihan Data

Tahap pertama adalah membersihkan data mentah lowongan kerja. Data mentah berupa berkas JSON berisi 20.298 lowongan yang memuat banyak nilai kosong dan format yang tidak konsisten. Kegiatan yang dilakukan meliputi tiga hal berikut.

Pertama, memilih baris yang memiliki daftar IT *Skills* non-kosong, karena hanya baris dengan daftar *skill* yang dapat digunakan untuk membangun label *multi-label*. Hasilnya, dari 20.298 lowongan mentah tersisa 4.667 baris data berkualitas, yaitu berkas JD2Skills_processed.csv.

Kedua, menangani nilai kosong (*missing values*). Hasil pemeriksaan menunjukkan kolom Soft *Skills* dan Education kosong pada seluruh baris (100%), sedangkan kolom Seniority kosong pada 18 baris. Karena kolom tersebut tidak digunakan sebagai target maupun fitur utama prediksi, kolom Soft *Skills* dan Education dihapus dari proses pemodelan, sedangkan nilai Seniority yang kosong tidak memengaruhi pelatihan.

Ketiga, memeriksa data duplikat. Hasil pemeriksaan menunjukkan tidak terdapat baris duplikat pada *dataset* hasil pembersihan, sehingga tidak diperlukan penghapusan duplikat tambahan. Contoh hasil pembersihan ditampilkan pada Tabel 3.5, yang menunjukkan perbandingan kondisi data sebelum dan sesudah pembersihan pada dua sampel baris.

Tabel 3.5 Hasil Pembersihan Data

Komponen	Sebelum (mentah)	Sesudah (JD2Skills_processed.csv)
Jumlah data	20.298 lowongan	4.667 baris
IT Skills kosong	Sebagian besar baris kosong	0 baris kosong
Education	Kosong seluruhnya	Tidak digunakan (dihapus)
Soft Skills	Kosong seluruhnya	Tidak digunakan (dihapus)
Kategori pekerjaan	Tidak konsisten	24 kategori baku

3.4.2. Normalisasi Skill

Tahap kedua adalah menormalkan penulisan *skill* agar konsisten. Kolom IT Skills pada *dataset* menyimpan banyak *skill* dalam satu sel yang dipisahkan koma, dengan variasi huruf besar/kecil dan spasi yang tidak seragam. Proses normalisasi dilakukan dengan mengubah seluruh *skill* menjadi huruf kecil (lowercase), menghapus spasi berlebih, dan memisahkan daftar *skill* berdasarkan koma. Sebagai contoh, nilai "Java, Java , python" pada sebuah baris diubah menjadi daftar *skill* ["java", "python"]. Normalisasi ini menjamin bahwa *skill* yang sama tidak dihitung sebagai dua entitas berbeda, sehingga frekuensi kemunculan dan representasi label menjadi konsisten.

3.4.3. Pembentukan Kandidat Skill (Label)

Tahap ketiga adalah membentuk daftar kandidat *skill* yang menjadi label pada model *multi-label*. Daftar kandidat diambil dari gabungan *skill* dengan

frekuensi kemunculan tertinggi pada *dataset*, kosakata *TF-IDF*, dan anotasi NER, kemudian diseleksi sebanyak 250 kandidat *skill* yang menjadi dimensi output model. Beberapa contoh kandidat *skill* yang menduduki frekuensi tertinggi adalah project management (2.390), sql (2.211), management (2.158), software development (2.020), dan java (1.913). Daftar 250 kandidat ini disimpan pada berkas `candidate_skills.json` dan menjadi label biner yang diprediksi oleh model.

3.4.4. Pembentukan Fitur TF-IDF

Tahap keempat adalah mengubah daftar *skill* pengguna menjadi representasi numerik menggunakan *TF-IDF*. Konteks *skill* pengguna (misalnya "python, sql, pandas") digabung menjadi satu dokumen, kemudian diubah menjadi vektor *TF-IDF* berdimensi 250 menggunakan vektorizer yang dilatih pada korpus seluruh daftar *skill* pada *dataset*. Sebagai contoh, dokumen "python, sql, pandas" menghasilkan vektor sparse dengan bobot *TF-IDF* tertinggi pada fitur python, sql, dan pandas, sedangkan fitur *skill* lain bernilai nol karena tidak muncul pada dokumen tersebut. Vektorizer ini disimpan pada berkas `tfidf_skills.pkl` agar dapat digunakan kembali saat prediksi.

3.4.5. Encoding Kategori Pekerjaan (One-Hot Encoding)

Tahap kelima adalah mengubah kategori pekerjaan target menjadi representasi *one-hot*. Sistem mendukung 24 kategori pekerjaan, sehingga setiap kategori direpresentasikan sebagai vektor biner berdimensi 24 dengan nilai 1 pada indeks kategori yang dipilih dan 0 pada indeks lainnya. Sebagai contoh, kategori "Data Science & AI" yang berada pada urutan kelima direpresentasikan sebagai vektor `[0, 0, 0, 0, 1, 0, ..., 0]` sepanjang 24 dimensi. Fitur *one-hot* ini digabungkan (*hstack*) dengan vektor *TF-IDF* sehingga total dimensi fitur model menjadi $250 + 24 = 274$ fitur.

3.4.6. Train-Test Split

Tahap terakhir adalah membagi *dataset* menjadi data latih dan data uji. Pembagian dilakukan dengan proporsi 80:20, yaitu sekitar 3.734 baris untuk data

latih dan sekitar 933 baris untuk data uji. Data latih digunakan untuk melatih model, sedangkan data uji digunakan untuk menghitung metrik evaluasi secara objektif. Selain split 80:20, dilakukan juga validasi silang *GroupKFold* 5-fold untuk menilai stabilitas performa model terhadap pembagian data yang berbeda. Konfigurasi fitur hasil seluruh tahap data preparation diringkas pada Tabel 3.6.

Tabel 3.6 Konfigurasi Fitur Model

Komponen	Nilai
Jumlah kandidat skill (label)	250 skill
Jumlah kategori pekerjaan	24 kategori
Jumlah fitur	274 fitur (250 TF-IDF + 24 one-hot)
Nilai default top-N	15 skill
Target model	Skill multi-label
Proporsi train-test split	80 : 20 (3.734 : 933 baris)

Berdasarkan Tabel 3.6, model tidak memprediksi satu kelas tunggal, tetapi memprediksi banyak kemungkinan *skill* yang dibutuhkan. Jumlah 274 fitur berasal dari gabungan fitur *TF-IDF* berbasis kandidat *skill* dan fitur kategori pekerjaan.

3.5 Modeling

Tahap Modeling bertujuan membangun model *multi-label skill* prediction. Model menerima input berupa *skill* pengguna dan target job category, kemudian menghasilkan probabilitas kebutuhan untuk setiap kandidat *skill*. Karena target berupa banyak label *skill*, pendekatan yang digunakan harus mampu menghasilkan beberapa label prediksi dalam satu proses inferensi.

Beberapa algoritma dibandingkan untuk memperoleh model yang paling sesuai dengan tujuan rekomendasi *top-N skill*. Model yang dibandingkan pada tahap modeling ditampilkan pada Tabel 3.7, dan masing-masing model beserta arsitekturnya dijelaskan pada subbab 3.5.1 sampai 3.5.3.

Tabel 3.7 Model Multi-Label yang Dibandingkan

No	Model	Keterangan
1	OneVsRest Logistic Regression	Model linear yang membangun satu classifier biner untuk setiap kandidat skill.
2	OneVsRest SGDClassifier	Model linear berbasis stochastic gradient descent untuk klasifikasi multi-label.
3	Random Forest Multi-Output	Model ensemble yang memprediksi beberapa label skill secara bersamaan.

3.5.1. OneVsRest Logistic Regression

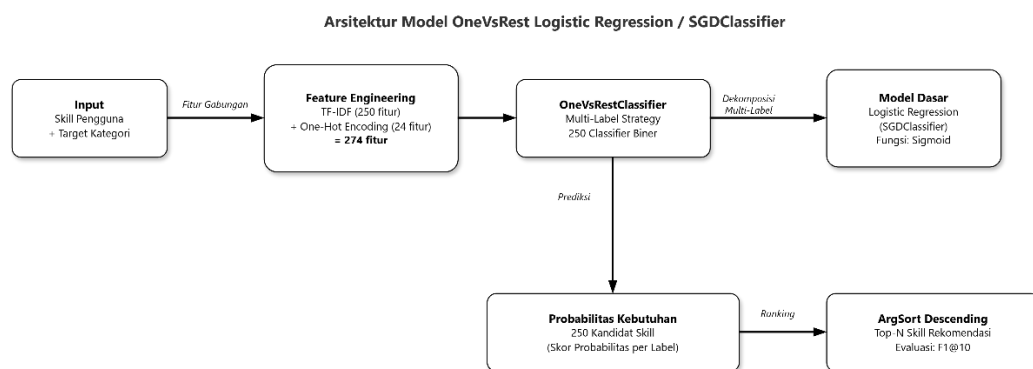
OneVsRest Logistic Regression menerapkan strategi binary relevance menggunakan Logistic Regression sebagai model dasar. Pada arsitektur ini, input berupa gabungan vektor *TF-IDF* 250 dimensi dan *one-hot* 24 dimensi (total 274 fitur) diumpankan ke dalam 250 classifier biner, di mana setiap classifier biner mempelajari probabilitas kebutuhan satu kandidat *skill* secara independen menggunakan fungsi sigmoid. Seluruh probabilitas dari 250 classifier kemudian digabungkan, dan *skill* dengan probabilitas tertinggi diambil sebagai daftar *top-N*. Model ini dipilih karena probabilistik, dapat diinterpretasikan, dan efisien untuk ruang label yang besar. Arsitektur model ditampilkan pada Gambar 3.4.

Pelatihan model dilakukan dengan membungkus *Logistic Regression* ke dalam *OneVsRestClassifier* sehingga dibentuk 250 *classifier* biner, yang masing-masing dilatih secara independen untuk mempelajari probabilitas kebutuhan satu kandidat *skill* menggunakan fungsi *loss logistik*. Seluruh *classifier* dilatih secara paralel pada 274 fitur yang sama, kemudian probabilitas keluaran setiap *classifier* digabungkan dan diurutkan untuk membentuk daftar *top-N skill*. Parameter konfigurasi model ditampilkan pada Tabel 3.8.

Tabel 3.8 Parameter OneVsRest Logistic Regression

Parameter	Nilai	Keterangan
C	1,0	Kebalikan kekuatan regularisasi L2; semakin kecil nilainya, regularisasi semakin kuat.

Parameter	Nilai	Keterangan
		Nilai 1,0 dipilih karena memberikan keseimbangan bias-varians terbaik berdasarkan validasi silang.
solver	liblinear	Algoritma optimasi yang efisien untuk data sparse berdimensi tinggi (274 fitur).
max_iter	2000	Batas maksimum iterasi agar model mencapai konvergensi.
n_jobs	-1	Pelatihan 250 classifier biner dijalankan secara paralel menggunakan seluruh core CPU.



Gambar 3.4 Arsitektur Model OneVsRest Logistic Regression / SGDClassifier

3.5.2. OneVsRest SGDClassifier

OneVsRest SGDClassifier menggunakan strategi yang identik dengan subbab 3.5.1, tetapi model dasarnya berupa *SGDClassifier* dengan fungsi loss logistik yang dilatih menggunakan optimasi stochastic gradient descent. Arsitekturnya ditunjukkan pada Gambar 3.4 dengan mengganti model dasar Logistic Regression menjadi *SGDClassifier*. Model ini efisien pada data berdimensi tinggi dan jarang (sparse), sehingga cocok untuk vektor *TF-IDF* berdimensi 274. Perbandingan keduanya dilakukan untuk mengetahui apakah metode optimasi yang berbeda memberikan peningkatan performa pada data lowongan kerja.

Pelatihan model dilakukan dengan strategi yang identik dengan subbab 3.5.1, yaitu membungkus *SGDClassifier* ke dalam *OneVsRestClassifier* sehingga setiap kandidat *skill* dipelajari oleh satu *classifier* biner. Perbedaanannya terletak pada metode optimasi: *SGDClassifier* menggunakan *stochastic gradient descent* dengan fungsi *loss logistik* yang memperbarui bobot model secara iteratif per sampel data, sehingga sangat efisien pada data berdimensi tinggi dan *sparse*. Pelatihan model ini menjadi yang tercepat di antara ketiga model yang dibandingkan. Parameter konfigurasi model ditampilkan pada Tabel 3.9.

Tabel 3.9 Parameter OneVsRest SGDClassifier

Parameter	Nilai	Keterangan
loss	log_loss	Fungsi loss logistik sehingga keluaran classifier dapat diinterpretasikan sebagai probabilitas kebutuhan skill.
alpha	0,0001	Koefisien regularisasi L2 yang mengontrol kompleksitas model; nilai 1e-4 dipilih berdasarkan validasi silang.
max_iter	2000	Jumlah epoch maksimum yang dilalui algoritma SGD hingga model konvergen.
random_state	42	Seed acak untuk menjamin hasil pelatihan dapat direproduksi.

3.5.3. Random Forest Multi-Output

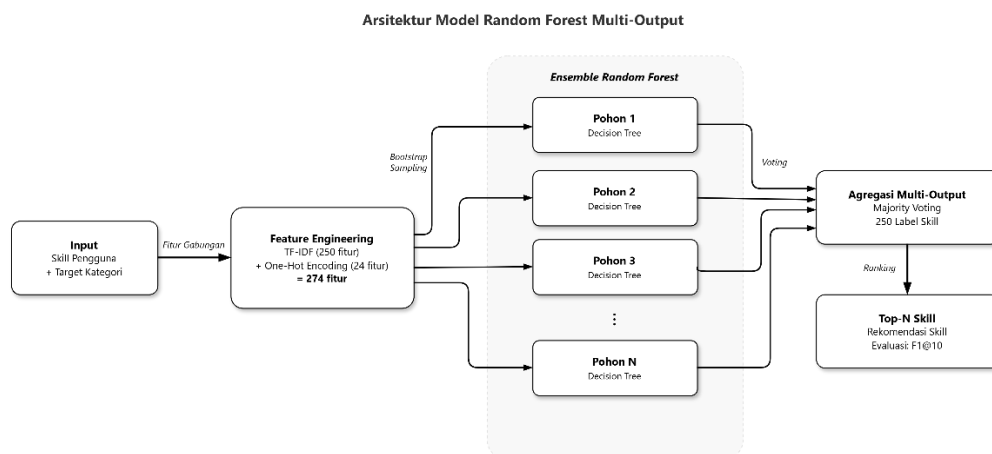
Random Forest *Multi-Output* membangun ensemble dari banyak pohon keputusan yang dilatih pada subset data acak, dan seluruh pohon memprediksi 250 label *skill* secara bersamaan melalui mekanisme *multi-output*. Hasil akhir diperoleh dari rata-rata probabilitas seluruh pohon untuk setiap label, kemudian diurutkan untuk membentuk daftar *top-N*. Berbeda dengan strategi *OneVsRest* yang melatih 250 classifier terpisah, Random Forest *Multi-Output* melatih satu ensemble yang memanfaatkan interaksi antar label dalam satu model. Arsitektur model ditampilkan pada Gambar 3.5.

Pelatihan model dilakukan dengan membangun satu *ensemble* berisi 200 pohon keputusan. Setiap pohon dilatih pada *bootstrap sample* acak dari data latih

dengan subset fitur acak, dan seluruh pohon memprediksi 250 label *skill* secara bersamaan melalui mekanisme *multi-output*. Hasil prediksi akhir diperoleh dari rata-rata probabilitas seluruh pohon untuk setiap label, kemudian diurutkan untuk membentuk daftar *top-N*. Karena membangun 200 pohon pada ruang label 250 *skill*, pelatihan model ini menjadi yang paling lama di antara ketiga model yang dibandingkan. Parameter konfigurasi model ditampilkan pada Tabel 3.10.

Tabel 3.10 Parameter Random Forest Multi-Output

Parameter	Nilai	Keterangan
n_estimators	200	Jumlah pohon keputusan dalam ensemble; nilai 200 memberikan keseimbangan antara akurasi dan waktu komputasi.
min_samples_leaf	2	Jumlah minimum sampel pada simpul daun untuk mencegah overfitting dan mengurangi varians.
random_state	42	Seed acak untuk menjamin hasil pelatihan dapat direproduksi.
n_jobs	-1	Pelatihan seluruh pohon dijalankan secara paralel menggunakan seluruh core CPU.



Gambar 3.5 Arsitektur Model Random Forest Multi-Output

3.5.4. Proses Pelatihan

Ketiga model dilatih menggunakan data latih hasil data preparation. Proses pelatihan mengikuti langkah berikut:

1. *skill* pengguna diubah menjadi vektor *TF-IDF* 250 dimensi
2. kategori pekerjaan target dikodekan *one-hot* 24 dimensi dan digabungkan dengan vektor *TF-IDF* menjadi 274 fitur
3. model dilatih untuk memprediksi probabilitas kebutuhan 250 kandidat *skill*
4. evaluasi dilakukan pada data uji serta validasi silang *GroupKFold* 5-fold.

Hasil evaluasi seluruh model dijelaskan pada subbab 3.6, dan model terbaik beserta komponennya disimpan setelah proses evaluasi selesai.

3.6 Evaluation

Tahap Evaluation dilakukan untuk menilai kualitas model *multi-label skill* prediction. Evaluasi dilakukan terhadap ketiga model yang telah dilatih pada subbab 3.5, kemudian hasilnya dibandingkan untuk menentukan model terbaik. Metrik evaluasi yang digunakan disesuaikan dengan output model berupa rekomendasi *top-N skill* dan ditampilkan pada Tabel 3.11.

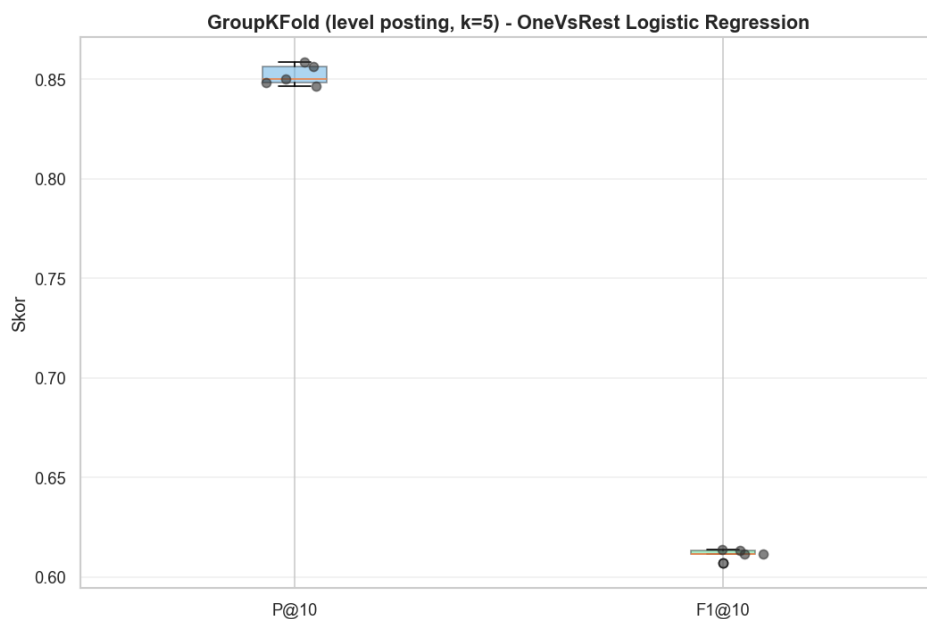
Tabel 3.11 Metrik Evaluasi Model

No	Metrik	Fungsi
1	Precision@K	Mengukur proporsi skill yang relevan dari daftar top-K skill yang direkomendasikan.
2	Recall@K	Mengukur proporsi skill relevan yang berhasil muncul dalam daftar top-K rekomendasi.
3	F1@K	Menggabungkan Precision@K dan Recall@K untuk menilai keseimbangan rekomendasi.
4	Hamming Loss	Mengukur rata-rata kesalahan prediksi label pada kasus multi-label.
5	Classification Report per Skill	Menunjukkan precision, recall, dan F1-score untuk setiap skill.
6	ROC-PR Curve	Menilai kemampuan model membedakan label skill dan kualitas prediksi probabilitas.

Selain metrik di atas, digunakan pula baseline majority untuk menilai apakah model memberikan peningkatan yang bermakna. Baseline majority memprediksi *skill* yang paling sering dibutuhkan pada setiap kategori pekerjaan, sehingga model yang baik harus mengungguli nilai $F1@10$ baseline tersebut.

3.6.1. Validasi Silang (K-Fold Cross Validation)

Sebelum membandingkan performa akhir, stabilitas model dinilai menggunakan validasi silang *GroupKFold* 5-fold. Pendekatan ini memastikan bahwa hasil evaluasi tidak bergantung pada satu pembagian data tertentu, sekaligus menilai kemampuan generalisasi model terhadap data yang belum pernah dilihat. Hasil validasi silang ditampilkan pada Gambar 3.6.



Gambar 3.6 K-Fold Cross Validation Model

Berdasarkan Gambar 3.6, nilai metrik antar fold relatif stabil untuk model terbaik, menunjukkan bahwa performa model tidak terlalu sensitif terhadap komposisi data latih dan data uji.

3.6.2. Hasil Evaluasi Ketiga Model

Ketiga model dievaluasi pada data uji menggunakan metrik *top-k*. Hasil evaluasi setiap model ditampilkan pada Tabel 3.12, sedangkan perbandingan precision, recall, dan *F1@10* antar model juga divisualisasikan pada Gambar 3.7, serta perbandingan precision-recall dan kurva precision-recall per nilai K ditampilkan pada Gambar 3.8 dan Gambar 3.9.

Tabel 3.12 Perbandingan Hasil Evaluasi Ketiga Model

Metrik	OneVsRest Logistic Regression	OneVsRest SGDClassifier	Random Forest (Multi-Output)	Baseline Majority
P@5	0,8938	0,8927	0,8690	0,5985
R@5	0,2513	0,2512	0,2414	0,1627
F1@5	0,3924	0,3921	0,3778	0,2559
P@10	0,8508	0,8488	0,8243	0,6001
R@10	0,4705	0,4690	0,4523	0,3256
F1@10	0,6059	0,6042	0,5841	0,4221
P@15	0,8029	0,8013	0,7780	0,4141
R@15	0,6582	0,6569	0,6349	0,3375
F1@15	0,7234	0,7219	0,6992	0,3719
Hamming Loss	0,0296	0,0302	0,0331	0,0653

Berdasarkan hasil evaluasi pada Tabel 3.12, terlihat bahwa nilai *Precision* (P@k) secara konsisten lebih tinggi dibandingkan nilai *Recall* (R@k) dan *F1-Score* (F1@k) pada seluruh nilai k yang diujikan (k=5, k=10, dan k=15). Fenomena ini disebabkan oleh perbedaan mendasar pada definisi kedua metrik tersebut dalam konteks evaluasi *top-k* pada permasalahan klasifikasi multi-label. Metrik P@k dihitung berdasarkan proporsi label relevan di antara k label dengan skor probabilitas tertinggi yang direkomendasikan oleh model, sehingga nilai penyebutnya bersifat tetap dan kecil (sebesar k). Sebaliknya, metrik R@k dihitung berdasarkan proporsi label relevan yang berhasil tercakup dalam k rekomendasi teratas terhadap keseluruhan jumlah label relevan (*ground truth*) pada setiap

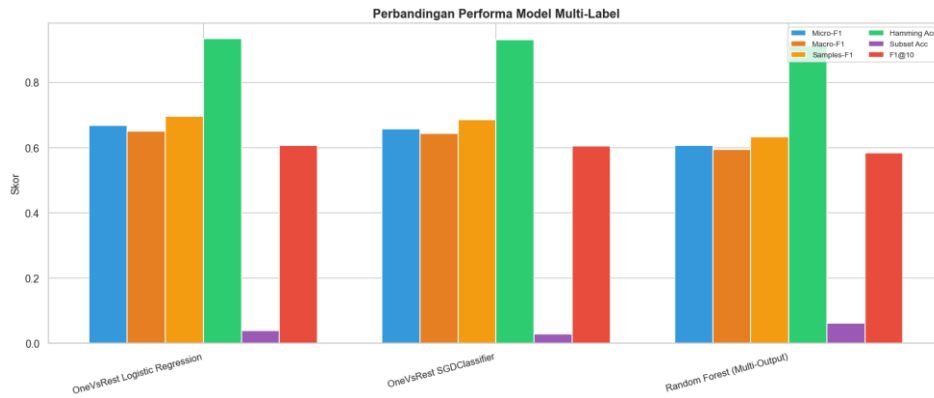
instance, di mana nilai penyebutnya dapat lebih besar dari k apabila rata-rata jumlah label positif per instance pada dataset melebihi nilai k yang digunakan.

Kondisi tersebut mengakibatkan adanya batas struktural (*structural ceiling*) pada nilai Recall, yaitu meskipun model mampu memberikan prediksi yang tepat pada seluruh k rekomendasi teratasnya, nilai Recall tidak akan mencapai nilai maksimal apabila jumlah label relevan pada suatu instance melebihi k . Hal ini didukung oleh tren peningkatan nilai $R@k$ secara signifikan seiring bertambahnya nilai k ($R@5$ sebesar 0,2513 meningkat menjadi $R@15$ sebesar 0,6582 pada model OneVsRest Logistic Regression), yang mengindikasikan bahwa semakin banyak slot rekomendasi yang disediakan, semakin besar pula peluang label-label relevan lainnya untuk tercakup dalam daftar rekomendasi.

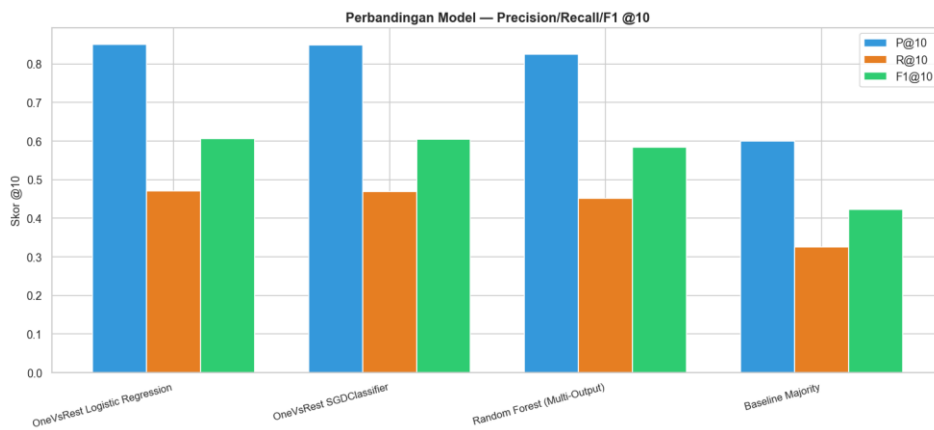
Di sisi lain, penurunan nilai $P@k$ seiring bertambahnya k (dari 0,8938 pada $P@5$ menjadi 0,8029 pada $P@15$) mengindikasikan bahwa model cenderung menempatkan prediksi dengan tingkat keyakinan (*confidence*) tertinggi pada peringkat teratas, sehingga label-label pada peringkat awal memiliki probabilitas lebih besar untuk benar. Semakin banyak label yang direkomendasikan, semakin besar pula kemungkinan tercakupnya label dengan tingkat keyakinan lebih rendah yang berpotensi tidak relevan, sehingga nilai Precision mengalami penurunan.

Selanjutnya, nilai F1-Score sebagai rata-rata harmonik (*harmonic mean*) dari Precision dan Recall cenderung lebih condong mendekati nilai yang lebih rendah di antara keduanya. Hal ini menjelaskan mengapa pada $k=5$, nilai $F1@5$ (0,3924) berada jauh lebih dekat dengan nilai $R@5$ (0,2513) dibandingkan dengan $P@5$ (0,8938), sebagai konsekuensi dari sifat rata-rata harmonik yang memberikan penalti lebih besar terhadap metrik dengan nilai lebih rendah. Semakin besar nilai k , selisih antara Precision dan Recall semakin mengecil, sehingga nilai F1-Score juga menunjukkan peningkatan yang konsisten ($F1@15$ sebesar 0,7234).

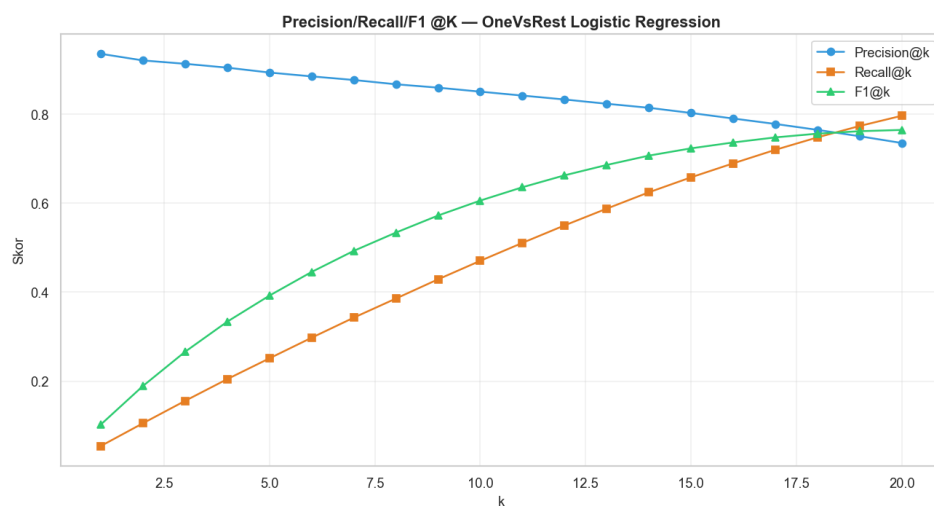
Secara keseluruhan, pola nilai Precision yang tinggi pada k kecil menunjukkan bahwa model memiliki tingkat akurasi yang baik dalam menghasilkan rekomendasi pada peringkat teratas, yang relevan dengan tujuan sistem ElVit dalam memberikan rekomendasi kompetensi yang paling sesuai dengan kebutuhan pengguna.



Gambar 3.7 Perbandingan Precision, Recall, dan F1@10 Antar Model



Gambar 3.8 Perbandingan Precision-Recall Antar Model



Gambar 3.9 Precision-Recall Kurva per Nilai K

3.6.3. Analisis Perbandingan Ketiga Model

Berdasarkan Gambar 3.7 sampai Gambar 3.9, ketiga model menunjukkan pola yang berbeda. Random Forest *Multi-Output* cenderung menghasilkan probabilitas yang lebih rata antar *skill* sehingga peringkat *top-N* kurang tajam, sementara model linear *OneVsRest* menghasilkan peringkat probabilitas yang lebih tegas. Pada metrik utama *F1@10*, *OneVsRest* Logistic Regression dan *OneVsRest SGDClassifier* menunjukkan hasil yang sebanding, dengan *OneVsRest* Logistic Regression memperoleh nilai yang lebih stabil pada validasi silang sehingga dijadikan model terpilih.

Secara khusus, *OneVsRest* Logistic Regression memperoleh nilai *F1@10* tertinggi di antara ketiga model. Secara keseluruhan, model linear berbasis *OneVsRest* (Logistic Regression dan *SGDClassifier*) menunjukkan performa yang lebih baik daripada Random Forest *Multi-Output* pada metrik berbasis peringkat, karena model linear lebih mampu memanfaatkan fitur *TF-IDF* yang jarang dan berdimensi tinggi untuk menghasilkan peringkat probabilitas yang akurat. Nilai numerik hasil evaluasi model final disajikan pada Tabel 3.13.

Pemilihan *OneVsRest* Logistic Regression sebagai model final didasarkan pada tiga pertimbangan. Pertama, nilai *F1@10* tertinggi di antara ketiga model. Kedua, sifat model yang probabilistik dan dapat diinterpretasikan, sehingga probabilitas kebutuhan setiap *skill* dapat digunakan langsung untuk menyusun peringkat rekomendasi yang transparan bagi pengguna. Ketiga, efisiensi inferensi yang tinggi pada ruang label 250 *skill*, sehingga latensi API tetap rendah pada aplikasi web. Random Forest *Multi-Output*, meskipun mampu menangkap interaksi antar label, tidak memberikan peningkatan yang cukup untuk mengkompensasi biaya komputasi yang lebih besar pada tahap deployment.

Tabel 3.13 Nilai Evaluasi Model Final (*OneVsRest* Logistic Regression)

No	Metrik	Nilai
1	P@5	0,8938
2	R@5	0,2513

No	Metrik	Nilai
3	F1@5	0,3924
4	P@10	0,8508
5	R@10	0,4705
6	F1@10	0,6059
7	P@15	0,8029
8	R@15	0,6582
9	F1@15	0,7234
10	Hamming Loss	0,0296
11	Baseline majority F1@10	0,4221

Berdasarkan Tabel 3.13, model final mencapai $F1@10$ sebesar 0,6059, meningkat 43,5% dibandingkan baseline majority (0,4221). Nilai $Precision@10$ sebesar 0,8508 menunjukkan bahwa sekitar 8 dari 10 *skill* yang direkomendasikan merupakan *skill* yang memang dibutuhkan pada kategori pekerjaan target, sedangkan $Recall@10$ sebesar 0,4705 menunjukkan bahwa model menangkap sekitar 47% dari seluruh *skill* relevan pada sepuluh rekomendasi pertama. Semakin besar nilai K, recall cenderung meningkat karena lebih banyak *skill* relevan yang tertangkap dalam daftar rekomendasi, namun precision sedikit menurun karena daftar rekomendasi yang lebih panjang meningkatkan kemungkinan munculnya *skill* yang kurang relevan.

Perlu dicatat bahwa hasil evaluasi juga menunjukkan keterbatasan model. Nilai true negative yang sangat besar dibandingkan true positive mengindikasikan bahwa model lebih dominan membedakan kelas negatif (*skill* yang tidak dibutuhkan), yang merupakan karakteristik umum permasalahan *multi-label* dengan ruang label besar dan proporsi label positif yang rendah. Kurva ROC yang tinggi (ROC-AUC) menunjukkan kemampuan model membedakan label positif dan negatif, namun nilai average precision pada kurva Precision-Recall (PR-AUC) yang lebih rendah menunjukkan bahwa kualitas peringkat rekomendasi *skill* positif masih dapat ditingkatkan

3.6.5. Komponen Model Final

Komponen model final disimpan setelah proses evaluasi selesai agar model terbaik yang telah terpilih dapat digunakan kembali pada backend *FastAPI*. Daftar komponen model final ditampilkan pada Tabel 3.14.

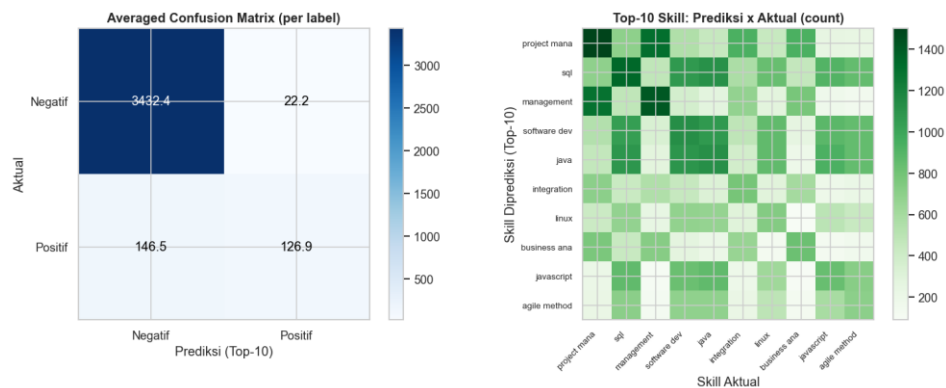
Tabel 3.14 Komponen Model Final

No	Komponen	Fungsi
1	skill_model.joblib	Menyimpan model OneVsRest Logistic Regression yang telah dilatih dan dievaluasi.
2	tfidf_skills.pkl	Menyimpan vectorizer TF-IDF untuk mengubah input skill menjadi fitur numerik.
3	candidate_skills.json	Menyimpan daftar 250 kandidat skill yang diprediksi model.
4	categories.json	Menyimpan daftar 24 kategori pekerjaan yang didukung sistem.
5	feature_config.json	Menyimpan konfigurasi jumlah skill, kategori, fitur, dan nilai top-N default.
6	final_model_info.json	Menyimpan metadata dan metrik evaluasi model final.

Berdasarkan Tabel 3.14, seluruh komponen yang digunakan oleh backend berada pada folder `final_model`. Komponen tersebut dimuat ketika aplikasi *FastAPI* dijalankan sehingga *endpoint* deteksi *gap* dan rekomendasi *skill* dapat menghasilkan prediksi secara konsisten.

3.6.4. Evaluasi Lanjutan pada Model Final

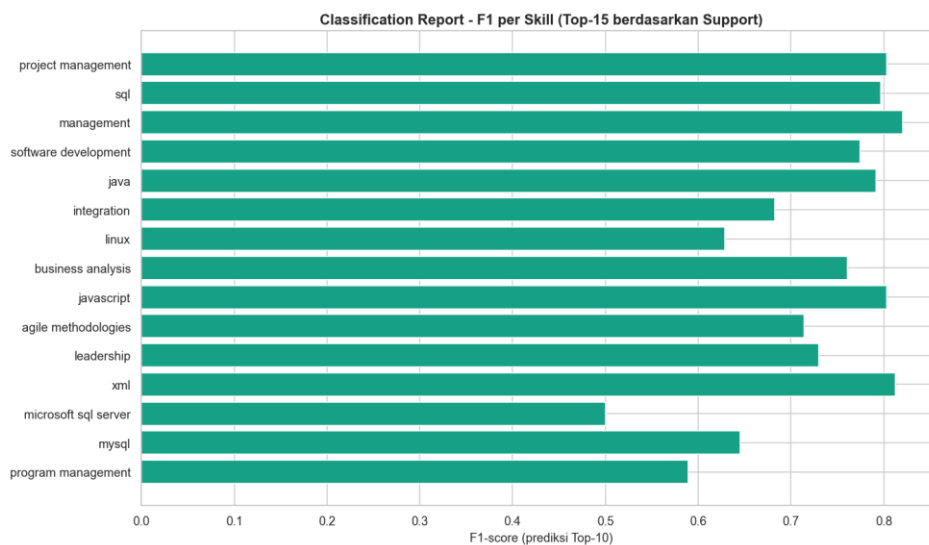
Evaluasi lanjutan dilakukan menggunakan confusion matrix *multi-label* untuk melihat pola prediksi benar dan salah pada level label *skill*. Confusion matrix *multi-label* ditampilkan pada Gambar 3.10.



Gambar 3.10 Confusion Matrix Multi-Label

Berdasarkan Gambar 3.10, confusion matrix menunjukkan bahwa jumlah true negative sangat besar dibandingkan true positive. Hal ini sesuai dengan karakteristik permasalahan *multi-label* 250 *skill*, di mana sebagian besar label memang tidak dibutuhkan pada suatu kategori pekerjaan tertentu. Analisis ini menegaskan pentingnya melihat metrik berbasis peringkat (top-k) sebagai ukuran utama, karena sistem menampilkan rekomendasi dalam bentuk daftar, bukan klasifikasi biner seluruh label.

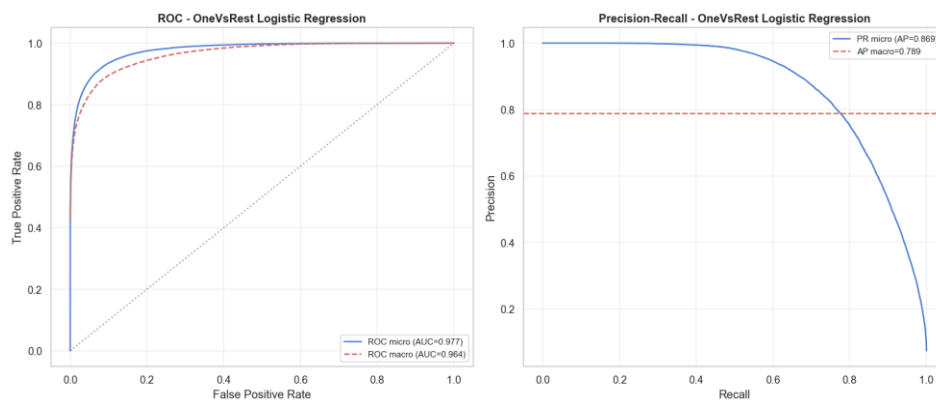
Classification report per *skill* digunakan untuk melihat performa model pada masing-masing *skill* yang memiliki support pada data uji. Hasil classification report per *skill* ditampilkan pada Gambar 3.11.



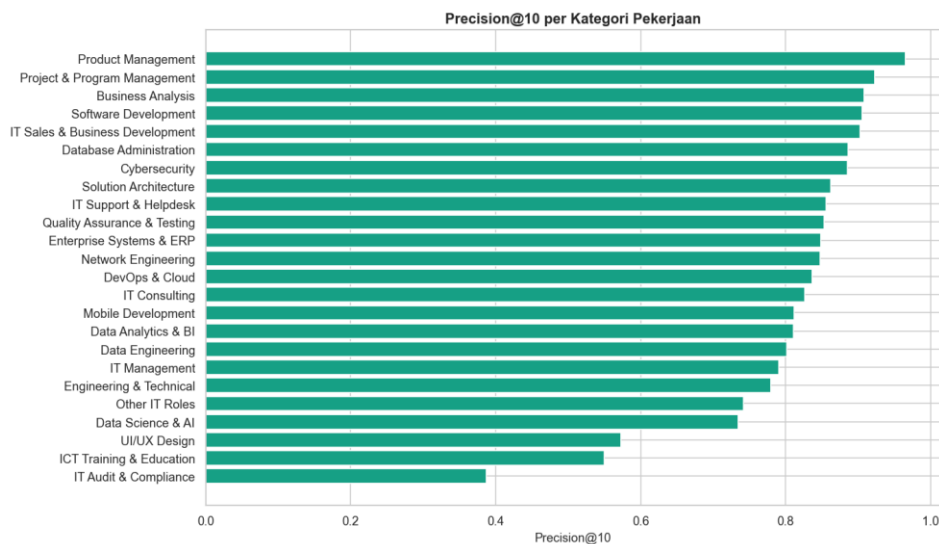
Gambar 3.11 Classification Report per Skill

Berdasarkan Gambar 3.11, performa setiap *skill* tidak sama karena frekuensi kemunculan dan pola keterkaitan *skill* pada *dataset* berbeda-beda. *Skill* dengan support lebih besar cenderung lebih mudah dipelajari oleh model, sedangkan *skill* yang jarang muncul menghasilkan nilai precision dan F1-score yang lebih rendah.

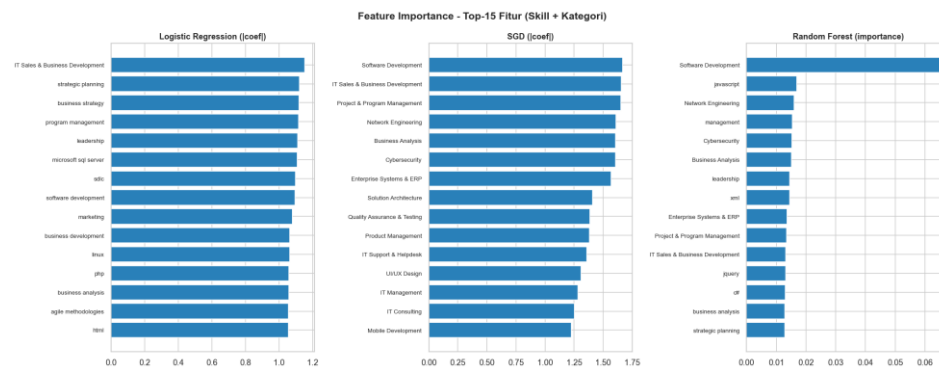
Evaluasi probabilitas model dilakukan menggunakan kurva ROC dan Precision-Recall. Kurva ROC-PR model terbaik ditampilkan pada Gambar 3.12, rata-rata *precision@10* per kategori pada Gambar 3.13, dan *feature importance* pada Gambar 3.14.



Gambar 3.12 ROC-PR Curve Model Terbaik



Gambar 3.13 Average Precision@10 per Kategori



Gambar 3.14 Feature Importance Semua Model

Berdasarkan Gambar 3.12, model memiliki kemampuan membedakan label *skill* pada tingkat tertentu; nilai ROC-AUC yang tinggi mengonfirmasi kemampuan pemisahan kelas, sementara nilai PR-AUC mengindikasikan kualitas peringkat rekomendasi yang masih dapat ditingkatkan. Gambar 3.13 menunjukkan bahwa *precision@10* bervariasi antar kategori, dan Gambar 3.14 memperlihatkan fitur-fitur dengan kontribusi terbesar terhadap prediksi. Hasil ini menjadi dasar bahwa pengembangan berikutnya perlu memperbaiki kualitas label *skill*, representasi fitur, dan metode pemodelan, sebagaimana disarankan pada Bab 4.

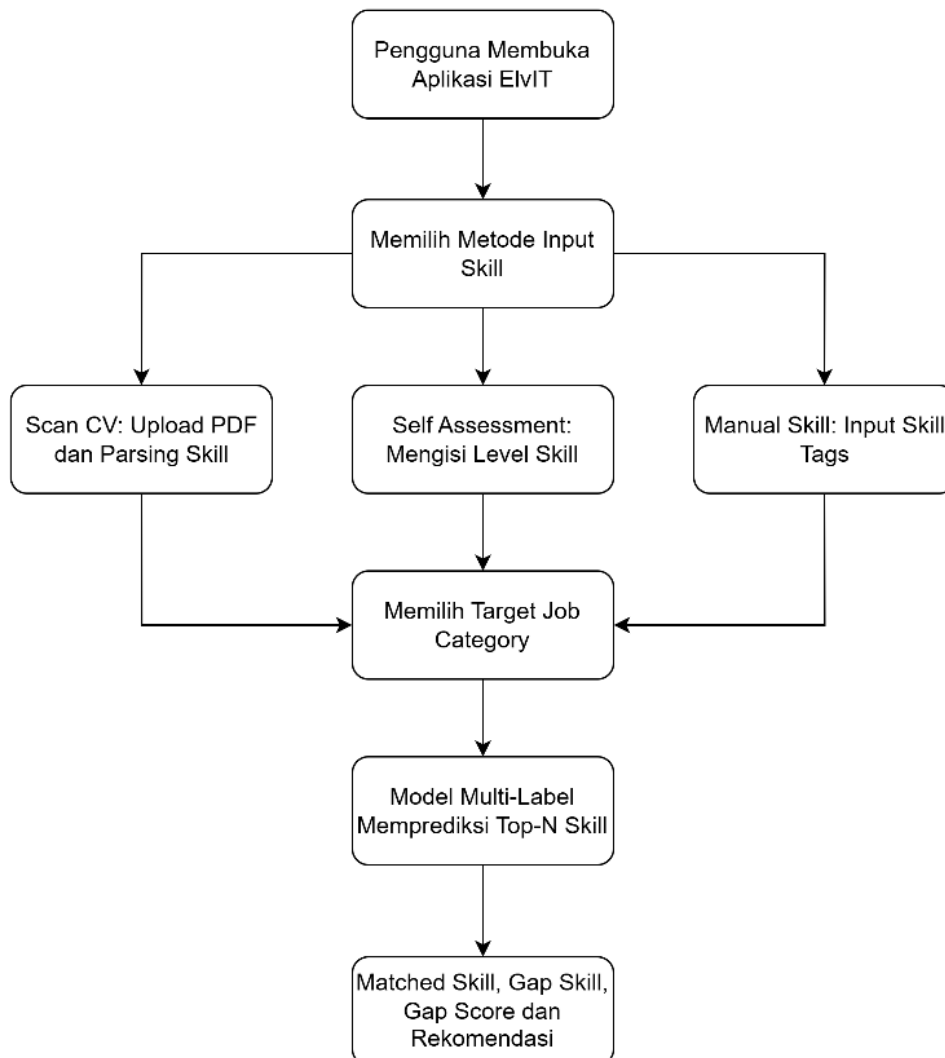
3.7 Deployment

Setelah model machine learning terpilih (OneVsRest Logistic Regression) dievaluasi dan komponennya disimpan, tahap deployment dilakukan untuk mengimplementasikan model tersebut ke dalam aplikasi *web* ElvIT dan menempatkannya pada lingkungan produksi agar dapat diakses secara online oleh pengguna melalui peramban web. Tahap deployment dimulai dari perancangan arsitektur deployment yang menjadi acuan implementasi, dilanjutkan dengan penyiapan komponen model, penempatan backend sebagai Application Programming Interface (API), penempatan frontend sebagai aplikasi statis, serta pengujian sistem secara keseluruhan.

3.7.1 Perancangan Sistem

Perancangan sistem ElvIT dibuat untuk menggambarkan alur interaksi pengguna dan proses pemrosesan data di dalam aplikasi. Alur utama sistem dimulai ketika pengguna memilih metode input *skill*, kemudian sistem memproses *skill* tersebut dan membandingkannya dengan kebutuhan target pekerjaan. Workflow sistem ElvIT ditampilkan pada Gambar 3.15.

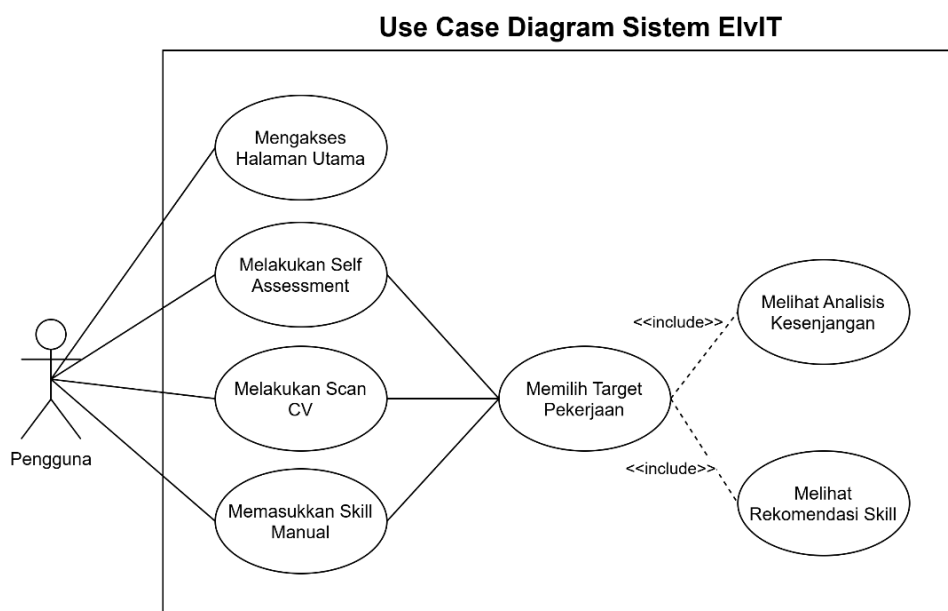
Workflow Diagram Sistem ElvIT



Gambar 3.15 Workflow Diagram Sistem ElvIT

Berdasarkan Gambar 3.15, sistem menyediakan tiga jalur input *skill*, yaitu Scan CV, Self Assessment, dan Manual *Skill*. Ketiga jalur tersebut menghasilkan daftar *skill* pengguna yang menjadi input bagi proses analisis berikutnya. Setelah pengguna memilih target job category, model *multi-label* memprediksi *top-N skill* yang relevan, kemudian sistem menentukan *skill* yang sudah sesuai dan *skill* yang masih menjadi gap.

Interaksi antara pengguna dan sistem juga digambarkan dalam use case diagram. Use case diagram digunakan untuk menunjukkan fungsi utama yang dapat dilakukan pengguna pada sistem ElvIT. Use case diagram sistem ditampilkan pada Gambar 3.16.



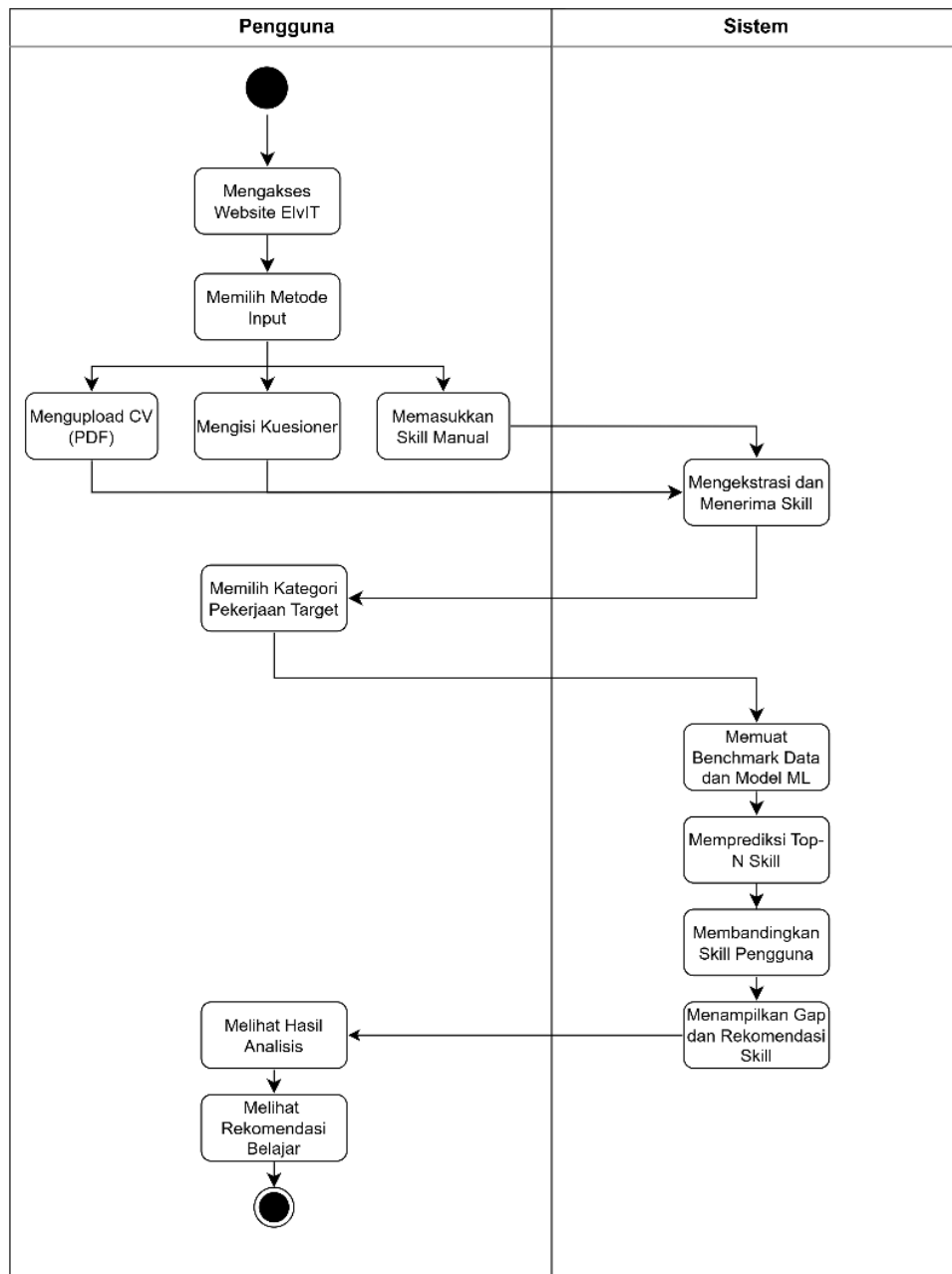
Gambar 3.16 Use Case Diagram Sistem ElvIT

Berdasarkan Gambar 3.16, pengguna dapat mengakses halaman utama, melakukan Scan CV, melakukan Self Assessment, memasukkan *skill* secara manual, memilih target pekerjaan, melihat hasil analisis kesenjangan, dan melihat rekomendasi *skill*. Use case tersebut disusun menggunakan kata kerja agar setiap fungsi merepresentasikan tindakan yang dilakukan pengguna terhadap sistem.

Aktivitas sistem secara lebih rinci digambarkan melalui activity diagram. Diagram ini menunjukkan urutan proses dari sisi pengguna dan sistem sejak awal

penggunaan aplikasi sampai hasil rekomendasi ditampilkan. Activity diagram sistem ElvIT ditampilkan pada Gambar 3.17.

Activity Diagram Sistem ElvIT



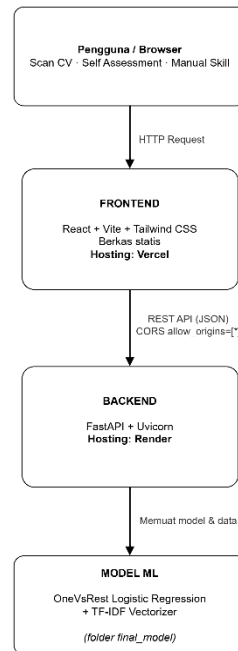
Gambar 3.17 Activity Diagram Sistem ElvIT

Berdasarkan Gambar 3.17, proses dimulai dari pengguna mengakses aplikasi dan memilih metode input. Sistem kemudian mengekstraksi atau menerima

daftar *skill*, memuat data kategori dan komponen model, memprediksi *skill* yang dibutuhkan, serta menampilkan hasil berupa gap *skill* dan rekomendasi *skill* yang perlu dipelajari.

3.7.2 Komponen Deployment

Aplikasi ElvIT dideploy dengan arsitektur client-server yang memisahkan frontend dan backend. Frontend dikembangkan menggunakan *React* dengan build tool *Vite* dan *Tailwind CSS*, kemudian dikompilasi menjadi berkas statis yang di-hosting pada platform *Vercel*. Backend dikembangkan menggunakan framework *FastAPI* dan berjalan pada platform *Render* melalui server *Uvicorn*. Seluruh permintaan deteksi kesenjangan *skill* dan rekomendasi dari frontend dikirimkan ke backend melalui endpoint REST API. Model machine learning *OneVsRest* Logistic Regression beserta *TF-IDF* vectorizer dimuat langsung oleh backend pada saat proses inisialisasi melalui fungsi *lifespan*, sedangkan data *benchmark skill* per kategori pekerjaan dimuat dari *dataset* yang telah diproses sebelumnya. Pipeline *NLP* berbasis *spaCy* juga diinisialisasi untuk mendukung fitur parsing CV. Komunikasi antara frontend dan backend diatur melalui konfigurasi CORS (Cross-Origin Resource Sharing) pada sisi server dengan parameter `allow_origins=["*"]` agar frontend yang di-hosting pada domain *Vercel* dapat mengakses API. Arsitektur deployment ditampilkan pada Gambar 3.18.



Gambar 3.18 Arsitektur Deployment Sistem ElvIT

3.7.3 Persiapan Komponen Deployment

Sebelum dilakukan deployment, seluruh komponen yang diperlukan disiapkan terlebih dahulu. Komponen model yang dideploy merupakan hasil pelatihan yang tersimpan pada folder `final_model/`, meliputi model *OneVsRest Logistic Regression* (`skill_model.joblib`), *TF-IDF* vectorizer (`tfidf_skills.pkl`), serta berkas konfigurasi berupa `candidate_skills.json`, `categories.json`, `feature_config.json`, dan `final_model_info.json`. Berkas `candidate_skills.json` berisi daftar 250 candidate *skills* yang memetakan indeks output model ke nama *skill*, sedangkan `categories.json` berisi 24 kategori pekerjaan IT yang didukung oleh sistem. Berkas `feature_config.json` menyimpan konfigurasi dimensi fitur (250 *skill*, 24 kategori, total 274 fitur) dan `final_model_info.json` menyimpan metadata model serta metrik evaluasi. Seluruh berkas konfigurasi tersebut digunakan oleh backend untuk memastikan konsistensi antara proses pelatihan dan proses prediksi terhadap data baru.

Pada sisi backend juga disiapkan berkas `requirements.txt` yang memuat seluruh dependensi Python, seperti *FastAPI*, *Uvicorn*, *scikit-learn*, *joblib*, *pandas*, *NumPy*, *Pydantic*, *PyMuPDF*, *spaCy*, dan *python-multipart*, serta skrip

render_build.sh yang menangani instalasi dependensi dan pengunduhan model *NLP* spaCy *en_core_web_sm*. Pada sisi frontend disiapkan berkas *vercel.json* untuk mengatur pola routing Single Page Application (SPA) serta variabel lingkungan *VITE_API_URL* yang menunjuk ke URL backend agar seluruh permintaan API dapat diarahkan dengan benar.

3.7.4 Deployment Backend

Backend API ElvIT dideploy pada platform cloud *Render*. Proses deployment dilakukan dengan menghubungkan repositori Git yang berisi kode backend ke layanan *Render*, kemudian menetapkan konfigurasi layanan melalui berkas *render.yaml*. Konfigurasi tersebut mendefinisikan layanan bertipe *web* dengan runtime Python versi 3.11.12 dan nama layanan *skill-gap-api*. Build command menggunakan skrip *render_build.sh* yang menjalankan tiga langkah: memperbarui pip, menginstal seluruh dependensi dari *requirements.txt*, dan mengunduh model bahasa spaCy *en_core_web_sm* yang dibutuhkan oleh pipeline *NLP* untuk ekstraksi *skill* dari dokumen CV. Start command menggunakan perintah *python run_api.py* yang menjalankan server *Uvicorn* dengan konfigurasi *uvicorn.run("api.main:app", host="0.0.0.0", port=port)*, di mana port dibaca dari variabel lingkungan *PORT* yang dialokasikan oleh platform *Render*. Isi skrip *render_build.sh* dan *run_api.py*, backend dapat diakses pada alamat <https://elvit.onrender.com> dan dokumentasi antarmuka API secara interaktif tersedia pada alamat <https://elvit.onrender.com/docs> melalui Swagger UI.

```
#!/usr/bin/env bash
set -o errexit

pip install --upgrade pip
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

```
import os
import uvicorn

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8000))
    uvicorn.run("api.main:app", host="0.0.0.0", port=port)
```

3.7.5 Deployment Frontend

Frontend aplikasi ElvIT dikembangkan menggunakan *React* dengan TypeScript dan di-build menjadi berkas statis. Proses build dilakukan dengan menjalankan perintah `npm run build` yang secara internal mengeksekusi `tsc` & `vite build`, yaitu melakukan type-checking TypeScript terlebih dahulu kemudian melakukan bundling dan minification menggunakan Vite sehingga menghasilkan folder `dist/` berisi berkas statis berupa HTML, CSS, dan JavaScript yang telah dioptimalkan. Hasil build terakhir menghasilkan bundle JavaScript utama berukuran sekitar 328 kB dan berkas CSS berukuran sekitar 61 kB. Folder hasil build tersebut kemudian di-hosting pada platform *Vercel* dengan langkah-langkah sebagai berikut.

Langkah pertama, kode frontend dipush ke repositori Git (GitHub) sehingga dapat diimpor oleh *Vercel*. Langkah kedua, repositori tersebut diimpor melalui dashboard *Vercel* (Import Project), kemudian dipilih framework preset Vite agar *Vercel* mengenali struktur proyek *React* otomatis. Langkah ketiga, konfigurasi build diverifikasi: build command diisi `npm run build` dan output directory diisi `dist`. Langkah keempat, variabel lingkungan `VITE_API_URL` diatur pada dashboard *Vercel* agar menunjuk ke alamat backend yang telah dideploy, yaitu `https://elvit.onrender.com`. Langkah kelima, proyek dideploy; *Vercel* menjalankan proses build dan menghasilkan URL produksi. Langkah keenam, berkas *vercel.json* memastikan seluruh rute ditangani oleh `index.html` sesuai pola Single Page Application (SPA), sehingga navigasi langsung ke halaman seperti `/self-assessment`, `/manual-input`, atau `/scan-cv` tidak menghasilkan error 404. Frontend yang telah dideploy dapat diakses oleh pengguna pada alamat `https://el-vit-three.vercel.app`.

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

3.7.6 Pengujian Sistem

Setelah seluruh komponen dideploy, dilakukan pengujian fungsional (black box) terhadap API dan alur integrasi frontend-backend untuk memastikan setiap fungsi sistem berjalan sesuai spesifikasi. Pengujian API dilakukan terhadap lima endpoint utama dan hasilnya ditampilkan pada Tabel 3.11.

Tabel 3. 15 Pengujian Endpoint API ElvIT

No	Endpoint	Skenario	Hasil yang Diharapkan	Hasil Aktual
1	GET /health	Memeriksa status layanan	Status ok dan jumlah kategori tersedia	Status ok, model termuat, 24 kategori (sukses)
2	GET /job-categories	Meminta daftar kategori pekerjaan	24 kategori pekerjaan	24 kategori dikembalikan (sukses)
3	POST /detect-gap	Input skill ["python","sql"], target "Data Science & AI"	Matched skills, gap skills, dan gap score	2 matched, 8 gap, gap_score 80,0 (sukses)
4	POST /recommend	Input skill dan target kategori	Daftar rekomendasi dengan priority rank	8 rekomendasi dengan priority rank 1-8 (sukses)
5	POST /parse-cv	Unggah PDF CV uji	Daftar skill yang tertulis pada CV	14 skill terdeteksi, seluruhnya tertulis pada CV (sukses)

Pengujian fitur pemindaian *CV* dilakukan dengan mengunggah PDF *CV* uji yang memuat daftar *skill* seperti python, sql, docker, kubernetes, terraform, aws, power bi, dan lainnya. Hasil deteksi pada Tabel 3.12 menunjukkan bahwa seluruh 14 *skill* yang terdeteksi benar-benar tertulis pada berkas CV, dan tidak ada *skill* yang tidak terdapat pada dokumen (false positive). Hasil ini diperoleh karena pipeline pencocokan *skill* menerapkan pencocokan eksak dengan normalisasi

bentuk jamak, sehingga kata dasar yang tidak tertulis pada *CV* tidak dianggap sebagai *skill*.

Tabel 3. 16 Pengujian Fitur Scan CV

No	Skenario	Hasil yang Diharapkan	Hasil Aktual
1	Unggah PDF CV uji yang memuat skill python, sql, docker, kubernetes, terraform, aws, gcp, power bi, pandas, airflow, etl, pipelines, apache, postgresql	Seluruh skill yang tertulis terdeteksi, tanpa skill palsu	14 skill terdeteksi, seluruhnya sesuai teks CV (sukses)
2	Unggah berkas non-PDF	Permintaan ditolak dengan pesan yang jelas	HTTP 400 dengan pesan "Only PDF files are supported" (sukses)
3	Unggah PDF dengan ukuran lebih dari 10 MB	Permintaan ditolak	HTTP 413 File too large (sukses)

Pengujian alur self assessment dan manual check dilakukan dengan mengirimkan daftar *skill* melalui endpoint `/detect-gap` dan `/recommend`, karena kedua fitur tersebut memanggil endpoint yang sama dengan skenario input yang berbeda. Hasil pengujian menunjukkan seluruh alur utama berjalan sesuai yang diharapkan, mulai dari penerimaan input, prediksi *top-N skill*, perbandingan *skill*, hingga penyusunan rekomendasi dengan priority rank. Selain itu, proses build frontend (`npm run build`) pada gambar 3.19 berhasil diselesaikan tanpa error, yang menandakan bahwa aplikasi produksi siap di-hosting pada *Vercel*.

```
PS C:\Users\Asus\prediction_skillv2\frontend> npm run build

> frontend@0.0.0 build
> tsc && vite build

vite v8.1.4 building client environment for production...
✓ 102 modules transformed.
computing gzip size...
dist/index.html                0.45 kB | gzip: 0.29 kB
dist/assets/elyit-icon-B7FXZdri.svg 77.75 kB | gzip: 32.73 kB
dist/assets/index-Cdbvg52P.css    45.80 kB | gzip: 9.15 kB
dist/assets/index-CgOp88gb.js    328.43 kB | gzip: 105.72 kB

✓ built in 297ms
PS C:\Users\Asus\prediction_skillv2\frontend>
```

Gambar 3.19 Hasil Uji Menggunakan `npm run build`

4. PENUTUP

4.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada sistem deteksi kesenjangan kompetensi bidang IT dalam aplikasi ElvIT, dapat disimpulkan bahwa penelitian ini telah berhasil membangun sebuah arsitektur sistem yang mampu memproses data profil profesional dan deskripsi pekerjaan secara otomatis menggunakan pendekatan NLP dan machine learning, dengan mengikuti kerangka kerja CRISP-DM serta menyediakan tiga jalur input kompetensi, yaitu Scan CV, Self Assessment, dan Manual Skill Check. Berdasarkan hasil perbandingan algoritma machine learning multi-label yang meliputi OneVsRest Logistic Regression, OneVsRest SGDClassifier, dan Random Forest Multi-Output, diperoleh bahwa OneVsRest Logistic Regression merupakan model yang paling optimal dalam memprediksi kebutuhan skill pada suatu kategori pekerjaan IT, dengan nilai $F1@10$ sebesar 0,6059, $Precision@10$ sebesar 0,8508, dan $Recall@10$ sebesar 0,4705, serta menunjukkan stabilitas yang baik pada validasi silang GroupKFold 5-fold.

Selanjutnya, hasil prediksi tersebut dimanfaatkan oleh modul rekomendasi keterampilan yang dipersonalisasi untuk menghasilkan daftar gap skill beserta rekomendasi skill prioritas berdasarkan gap score, match score, dan priority rank, sehingga pengguna memperoleh panduan pengembangan keterampilan yang terarah. Sistem prediksi dan rekomendasi ini juga telah berhasil diintegrasikan ke dalam platform aplikasi web yang interaktif dengan backend FastAPI dan frontend React, sehingga memungkinkan pengguna untuk memasukkan data kompetensi mereka secara mandiri dan memperoleh umpan balik berupa hasil deteksi kesenjangan kompetensi serta rekomendasi pengembangan skill secara langsung. Meskipun evaluasi menyeluruh menunjukkan ketepatan rekomendasi yang tergolong baik, nilai $recall@10$ dan PR-AUC yang masih relatif rendah mengindikasikan bahwa belum seluruh skill relevan tertangkap pada sepuluh rekomendasi pertama, dengan cakupan sistem yang masih terbatas pada 24 kategori pekerjaan dan 250 kandidat skill dari satu sumber data. Secara keseluruhan, aplikasi

ElvIT terbukti dapat dimanfaatkan sebagai alat bantu awal yang efektif guna membantu tenaga profesional maupun calon pekerja di bidang IT dalam memahami kesenjangan kompetensi yang dimiliki serta mengarahkan pengembangan keterampilan mereka secara lebih terarah dan personal.

4.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemukan, beberapa saran dikemukakan untuk pengembangan lebih lanjut. *Dataset* yang digunakan dapat diperluas dengan menambahkan data lowongan kerja yang lebih baru dan lebih beragam, serta mengintegrasikan data dari sumber lain seperti Indeed atau situs karier perusahaan, untuk meningkatkan representasi dan relevansi *benchmark skill* industri.

Model klasifikasi dapat dikembangkan lebih lanjut dengan menerapkan model berbasis BERT atau IndoBERT sebagai alternatif representasi fitur teks, terutama untuk menangani variasi semantik dalam penulisan *skill* yang tidak dapat ditangkap sepenuhnya oleh TF-IDF.

Sistem rekomendasi dapat ditingkatkan dengan menambahkan modul *collaborative filtering* atau *knowledge graph* berbasis ontologi SKKNI/SFIA untuk menghasilkan jalur pembelajaran (*learning path*) yang lebih terstruktur dan personal bagi pengguna.

Aplikasi ElvIT dapat dikembangkan lebih lanjut dengan menambahkan fitur autentikasi pengguna, riwayat analisis, dan integrasi dengan platform pembelajaran daring (seperti Coursera, Udemy, atau Dicoding) untuk langsung menghubungkan rekomendasi *skill* dengan sumber belajar yang relevan.

DAFTAR PUSTAKA

- 1.5. *Stochastic Gradient Descent* — *scikit-learn 1.9.0 documentation*. (n.d.). Retrieved August 19, 2026, from <https://scikit-learn.org/stable/modules/sgd.html>
- Arslan, M., & Cruz, C. (2023). *Imbalanced Multi-label Classification for Business-related Text with Moderately Large Label Spaces*. <https://arxiv.org/pdf/2306.07046>
- Bhola, A., Halder, K., Prasad, A., & Kan, M. Y. (2020). *WING-NUS/JD2Skills-BERT-XMLC: Code and Dataset for the Bhola et al. (2020) Retrieving Skills from Job Descriptions: A Language Model Based Extreme Multi-label Classification Framework*. <https://github.com/WING-NUS/JD2Skills-BERT-XMLC>
- Bowers, D., & Sabin, M. (2022). Using a Professional Skills Framework to Support the Assessment of Dispositions in IT Education. *SIGITE 2022 - Proceedings of the 23rd Annual Conference on Information Technology Education*, 103–109. <https://doi.org/10.1145/3537674.3554747>
- Braun, G., Rätty, P., Bokinge, M., Rikala, P., Hämäläinen, R., Syberfeldt, A., & Stahre, J. (2025). The skill bridge – A global qualitative analysis of skill gap management. *Journal of Environmental Management*, 395, 127738. <https://doi.org/10.1016/J.JENVMAN.2025.127738>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324/METRICS>
- Cedefop. (2018). *Insights into skill shortages and skill mismatch Learning from Cedefop's European skills and jobs survey*.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0 Step-by-step data mining guide*.
- Classification of text documents using sparse features* — *scikit-learn 1.9.0 documentation*. (n.d.). Retrieved August 19, 2026, from https://scikit-learn.org/stable/auto_examples/text/plot_document_classification_20newsgroups.html
- Deldjoo, Y., He, Z., Mcauley, J., Korikov, A., Sanner, S., Ramisa, A., Vidal, R., Sathiamoorthy, M., Kasrizadeh, A., Milano, S., Ricci, F., Heezemans, A., &

- Casey, M. (2024). *Recommendation with Generative Models*. XX, No. XX, 1–120. <https://arxiv.org/pdf/2409.15173>
- Diniz, W., Valença, M., França, C., Santos, A., & Pincovsky, M. (2024). The Skill Gap in Software Industry: A Mapping Study. *Simpósio Brasileiro de Engenharia de Software (SBES)*, 1, 192–200. <https://doi.org/10.5753/SBES.2024.3351>
- Elia, D., Chen, F., Zowghi, D., & Rizoiu, M.-A. (2023). *Australasian Conference on Information Systems The Innovation-to-Occupations Ontology: Linking Business Transformation Initiatives to Occupations and Skills*.
- Fatima, S., Hussain, A., Amir, S. Bin, Awan, M. G. Z., Ahmed, S. H., & Aslam, S. M. H. (2023). XGBoost and Random Forest Algorithms: An in Depth Analysis. *Pakistan Journal of Scientific Research*, 3(1), 26–31. <https://doi.org/10.57041/VOL3ISS1PP26-31>
- Framesthi, D., Keizer, H., Kesumah, P., Subrayanti, D., Firmansyah, Y., Nafisah, Z., & Rohaeni, N. (2025). ANALISIS KESENJANGAN KOMPETENSI ANGKATAN KERJA MUDA KOTA CIMAHI TERHADAP KEBUTUHAN INDUSTRI 4.0. *JURNAL EKONOMI BISNIS DAN MANAJEMEN (EKO-BISMA)*, 4, 426–434. <https://doi.org/10.58268/eb.v4i2.227>
- Goyal, N., Kalra, J. S., Sharma, C., Mutharaju, R., Sachdeva, N., & Kumaraguru, P. (2023). JobXMLC: EXtreme Multi-Label Classification of Job Skills with Graph Neural Networks. *EACL 2023 - 17th Conference of the European Chapter of the Association for Computational Linguistics, Findings of EACL 2023*, 2181–2191. <https://doi.org/10.18653/V1/2023.FINDINGS-EACL.163>
- He, J., Xu, B., Yang, Z., Han, D., Yang, C., Liu, J., Zhao, Z., Lo, D., He, J., Yang, Z., Yang, C., Liu, J., Zhao, Z., Lo, D., Xu, B., & Han, D. (2024). *PTM4Tag+: Tag Recommendation of Stack Overflow Posts with Pre-trained Models*. <https://stackexchange.com/sites?view=list#traffic>
- How SFIA works - Levels of responsibility and skills — English*. (n.d.). Retrieved August 18, 2026, from <https://sfia-online.org/en/about-sfia/how-sfia-works>
- Jena, K. K., Bhoi, S. K., Malik, T. K., Sahoo, K. S., Jhanjhi, N. Z., Bhatia, S., & Amsaad, F. (2023). E-Learning Course Recommender System Using Collaborative Filtering Models. *Electronics (Switzerland)*, 12(1). <https://doi.org/10.3390/electronics12010157>
- Jiechieu, K. F. F., & Tsopze, N. (2020). Skills prediction based on multi-label resume classification using CNN with model predictions explanation. *Neural Computing and Applications* 2020 33:10, 33(10), 5069–5087. <https://doi.org/10.1007/S00521-020-05302-X>

- Kementerian Komunikasi dan Informatika Republik Indonesia. (2023). *Buku Visi Indonesia Digital 2045 - Indonesia Digital 2045*. <https://digital2045.id/bukuvid2045/>
- Kementerian Perindustrian Republik Indonesia. (2018). *Making Indonesia 4.0*.
- Keputusan Menteri Ketenagakerjaan Nomor 30 Tahun 2025. (2025). <https://jdih.kemnaker.go.id/peraturan/detail/2634/keputusan-menteri-nomor-30-tahun-2025>
- Khaouja, I., Kassou, I., & Ghogho, M. (2021). A Survey on Skill Identification from Online Job Ads. *IEEE Access*, 9, 118134–118153. <https://doi.org/10.1109/ACCESS.2021.3106120>
- Kistianingrum, F., Khoirunnisa, I., Fitriyani, S., & Mulyeni, S. (2026). Analisis Kesenjangan Keterampilan Lulusan S1 dengan Kebutuhan Industri 4.0 dan 5.0 di Indonesia. *RISOMA : Jurnal Riset Sosial Humaniora Dan Pendidikan*, 4(2), 43–54. <https://doi.org/10.62383/RISOMA.V4I2.1552>
- Lentzas, A., Dalagdi, E., & Vrakas, D. (2022). Multilabel Classification Methods for Human Activity Recognition: A Comparison of Algorithms. *Sensors* 2022, Vol. 22, Page 2353, 22(6), 2353. <https://doi.org/10.3390/S22062353>
- Leon, F., Gavrilescu, M., Floria, S. A., & Minea, A. A. (2024). Hierarchical Classification of Transversal Skills in Job Advertisements Based on Sentence Embeddings. *Information (Switzerland)*, 15(3). <https://doi.org/10.3390/INFO15030151>
- Liddy, E. D. (2001). Natural Language Processing. *School of Information Studies - Faculty Scholarship*. <https://surface.syr.edu/istpub/63>
- Putra, H., & Rumini, R. (2025). Comparative Study of Logistic Regression, Random Forest, and XGBoost for Bank Loan Approval Classification. *Journal of Applied Informatics and Computing*, 9(5), 2822–2835. <https://doi.org/10.30871/JAIC.V9I5.10862>
- Schröer, C., Kruse, F., & Gómez, J. M. (2021). A Systematic Literature Review on Applying CRISP-DM Process Model. *Procedia Computer Science*, 181, 526–534. <https://doi.org/10.1016/J.PROCS.2021.01.199>
- Senger, E., Zhang, M., van der Goot, R., & Plank, B. (2024). Deep Learning-based Computational Job Market Analysis: A Survey on Skill Extraction and Classification from Job Postings. *NLP4HR 2024 - 1st Workshop on Natural Language Processing for Human Resources, Proceedings of the Workshop*, 1–15. <https://doi.org/10.18653/V1/2024.NLP4HR-1.1>

- Setiawan, Y., Gunawan, D., & Efendi, R. (2022). Feature Extraction TF-IDF to Perform Cyberbullying Text Classification: A Literature Review and Future Research Direction. *2022 International Conference on Information Technology Systems and Innovation, ICITSI 2022 - Proceedings*, 283–288. <https://doi.org/10.1109/ICITSI56531.2022.9970942>
- Sinha, A. K., Akhtar, M. A. K., & Kumar, M. (2023). Automated Resume Parsing and Job Domain Prediction using Machine Learning. *Indian Journal of Science and Technology*, 16(26), 1967–1974. <https://doi.org/10.17485/IJST/V16I26.880>
- skill gap* | CEDEFOP. (2023). <https://www.cedefop.europa.eu/en/tools/vet-glossary/glossary/deficit-de-competence>
- Sobirin, R. (2020). Pendidikan Teknik Berbasis SKKNI pada Mata Kuliah Kewirausahaan di Teknik Elektro. *Teknologi Nusantara*, 2(1). <https://doi.org/10.30999/TEKNU.V2I1.2381>
- Ternikov, A. (2022). Soft and hard skills identification: Insights from IT job advertisements in the CIS region. *PeerJ Computer Science*, 8, e946. <https://doi.org/10.7717/PEERJ-CS.946/SUPP-1>
- The Future of Jobs Report 2020* | World Economic Forum. (2020). <https://www.weforum.org/publications/the-future-of-jobs-report-2020/>
- Vermeer, N. (2022). Using RobBERT and eXtreme Multi-Label Classification to Extract Implicit and Explicit Skills From Dutch Job Descriptions. *Compjobs '22: Computational Jobs Marketplace*, Feb 25, 2022, 1. <https://www.textkernel.com/nl/>
- Yunitarini, R., Pradita, D. A., & Widiaswanti, E. (2025). Implementation of term frequency-inverse document frequency (TF-IDF) and Word2Vec in traditional medicine recommendation system based on content-based filtering. *Eastern-European Journal of Enterprise Technologies*, 4(2 (136)), 70–80. <https://doi.org/10.15587/1729-4061.2025.338128>
- Zhu, W., Qiu, R., & Fu, Y. (2024). *Comparative Study on the Performance of Categorical Variable Encoders in Classification and Regression Tasks*. <https://arxiv.org/pdf/2401.09682>

LAMPIRAN

Lampiran 1 Listing Program – API Backend (main.py)

File main.py merupakan entry point aplikasi *backend* ElvIT yang dibangun menggunakan *framework* FastAPI. File ini mendefinisikan seluruh *endpoint* REST API, konfigurasi CORS, serta lifecycle aplikasi yang memuat model *machine learning multi-label* dan data pekerjaan pada saat startup.

```
from fastapi import FastAPI, HTTPException, UploadFile, File

from fastapi.middleware.cors import CORSMiddleware

from fastapi.staticfiles import StaticFiles

from fastapi.responses import FileResponse

from contextlib import asynccontextmanager

import os


from .services.model_service import load_artifacts, predict_skills

from .services.data_service import load_jobs_data, build_job_category_skills,
get_available_categories

from .services.cv_service import extract_text_from_pdf, extract_skills_from_text

from .schemas import (
    GapDetectRequest, GapDetectResponse, SkillItem,
    RecommendRequest, RecommendResponse, RecommendItem,
    HealthResponse, CVParseResponse
)


@asynccontextmanager
async def lifespan(app: FastAPI):
    load_artifacts()
    df = load_jobs_data()
    app.state.job_category_skills = build_job_category_skills(df)
    app.state.available_categories =
get_available_categories(app.state.job_category_skills)
    yield


app = FastAPI(
    title="Skill Gap Detection API",
    description="Deteksi kesenjangan kompetensi profesional IT berbasis model multi-label skill prediction",
    version="2.0.0",
```

```

        lifespan=lifespan
    )

# — CORS (allow browser requests from any origin, e.g. Vite dev server) ———
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# — Static HTML pages (celah-*.html) served at root level —————
_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

@app.get("/celah-cv", include_in_schema=False)
@app.get("/celah-cv.html", include_in_schema=False)
def serve_celah_cv():
    path = os.path.join(_ROOT, "celah-cv.html")
    if os.path.exists(path):
        return FileResponse(path, media_type="text/html")
    raise HTTPException(status_code=404, detail="celah-cv.html not found")

@app.get("/celah-mockup", include_in_schema=False)
@app.get("/celah-mockup-minimal.html", include_in_schema=False)
def serve_celah_mockup():
    path = os.path.join(_ROOT, "celah-mockup-minimal.html")
    if os.path.exists(path):
        return FileResponse(path, media_type="text/html")
    raise HTTPException(status_code=404, detail="celah-mockup-minimal.html not found")

@app.get("/health", response_model=HealthResponse, tags=["System"])
def health():
    return HealthResponse(
        status="ok",
        model_loaded=True,
        categories_available=len(app.state.available_categories)
    )

```

```

    )

@app.get("/job-categories", tags=["Information"])
def list_categories():
    return {
        "count": len(app.state.available_categories),
        "categories": app.state.available_categories
    }

@app.post("/detect-gap", response_model=GapDetectResponse, tags=["Skill Gap"])
def detect_gap(req: GapDetectRequest):
    result = predict_skills(req.skills, req.target_job, req.top_n)
    if result is None:
        raise HTTPException(
            status_code=404,
            detail=f"Job category '{req.target_job}' not found. Available:
{app.state.available_categories}"
        )
    return GapDetectResponse(
        target_job=result['target_job'],
        benchmark_count=result['benchmark_count'],
        matched=[SkillItem(**m) for m in result['matched']],
        gap=[SkillItem(**g) for g in result['gap']],
        gap_score=result['gap_score'],
        match_score=result['match_score'],
    )

@app.post("/recommend", response_model=RecommendResponse, tags=["Skill Gap"])
def recommend(req: RecommendRequest):
    result = predict_skills(req.skills, req.target_job, req.top_n)
    if result is None:
        raise HTTPException(
            status_code=404,
            detail=f"Job category '{req.target_job}' not found. Available:
{app.state.available_categories}"

```

```

    )
    return RecommendResponse(
        target_job=result['target_job'],
        gap_score=result['gap_score'],
        match_score=result['match_score'],
        recommendations=[RecommendItem(**r) for r in result['recommended']],
    )

@app.post("/parse-cv", response_model=CVParseResponse, tags=["CV Scanner"])
async def parse_cv(file: UploadFile = File(...)):
    """Upload a PDF CV and get detected skills via NLP pipeline."""
    if not file.filename or not file.filename.lower().endswith(".pdf"):
        raise HTTPException(
            status_code=400,
            detail="Only PDF files are supported. Please upload a .pdf file."
        )

    content_type = file.content_type or ""
    if content_type and "pdf" not in content_type.lower():
        raise HTTPException(
            status_code=400,
            detail=f"Invalid content type '{content_type}'. Expected application/pdf."
        )

    try:
        file_bytes = await file.read()
    except Exception as exc:
        raise HTTPException(status_code=500, detail=f"Failed to read uploaded file: {exc}")

    if len(file_bytes) > 10 * 1024 * 1024: # 10 MB limit
        raise HTTPException(status_code=413, detail="File too large. Maximum size is 10 MB.")

    try:
        raw_text = extract_text_from_pdf(file_bytes)
    except ValueError as exc:

```



```

        raise HTTPException(status_code=422, detail=str(exc))
    except RuntimeError as exc:
        raise HTTPException(status_code=503, detail=str(exc))

    # Collect all known skills across every job category
    all_known_skills: list = []
    for cat_skills in app.state.job_category_skills.values():
        all_known_skills.extend(cat_skills.keys())

    result = extract_skills_from_text(raw_text, all_known_skills)

    return CVParseResponse(**result)

```

Lampiran 2 *Listing* Program Schemas API (schemas.py)

File `schemas.py` mendefinisikan seluruh model data (Pydantic `BaseModel`) yang digunakan sebagai kontrak *request* dan *response* pada setiap *endpoint* API. Penggunaan Pydantic memastikan validasi data otomatis dan dokumentasi OpenAPI yang dihasilkan secara otomatis oleh FastAPI.

```

from pydantic import BaseModel, Field
from typing import List, Optional

class GapDetectRequest(BaseModel):
    skills: List[str] = Field(..., description="Daftar skill yang dimiliki user")
    target_job: str = Field(..., description="Job category target")
    top_n: Optional[int] = Field(15, description="Jumlah top skill yang diprediksi model")

class SkillItem(BaseModel):
    skill: str
    prob: float = Field(..., description="Probabilitas model bahwa skill dibutuhkan untuk target job")

class GapDetectResponse(BaseModel):
    target_job: str
    benchmark_count: int
    matched: List[SkillItem]
    gap: List[SkillItem]
    gap_score: float

```

```

    match_score: float

class RecommendRequest(BaseModel):
    skills: List[str] = Field(..., description="Daftar skill yang dimiliki user")
    target_job: str = Field(..., description="Job category target")
    top_n: Optional[int] = Field(15, description="Jumlah top skill yang diprediksi model")

class RecommendItem(BaseModel):
    skill: str
    prob: float = Field(..., description="Probabilitas model bahwa skill dibutuhkan")
    priority_rank: int = Field(..., description="Urutan prioritas belajar (1 = paling penting)")

class RecommendResponse(BaseModel):
    target_job: str
    gap_score: float
    match_score: float
    recommendations: List[RecommendItem]

class HealthResponse(BaseModel):
    status: str = "ok"
    model_loaded: bool
    categories_available: int

class CVParseResponse(BaseModel):
    detected_skills: List[str] = Field(..., description="Daftar skill yang terdeteksi dari CV")
    skill_count: int = Field(..., description="Jumlah skill yang berhasil terdeteksi")
    raw_text_preview: str = Field("", description="Preview 500 karakter pertama teks CV")
    extraction_notes: str = Field("", description="Info pipeline yang digunakan")

```

Lampiran 3 Listing Program Model Service (model_service.py)

File `model_service.py` bertanggung jawab memuat komponen model *machine learning multi-label* yang telah dilatih, yaitu model prediksi *skill*, vektorizer TF-IDF, daftar kandidat *skill*, dan daftar kategori pekerjaan. File ini juga

menyediakan fungsi prediksi *top-N skill* yang dibutuhkan suatu kategori pekerjaan dengan mengombinasikan representasi TF-IDF dari *skill* pengguna dan *one-hot encoding* kategori pekerjaan.

```
import os

import json

import joblib

import numpy as np

from scipy.sparse import hstack, csr_matrix


BASE_DIR = os.path.dirname(os.path.dirname(os.path.dirname(__file__)))
FINAL_DIR = os.path.join(BASE_DIR, 'final_model')
MODEL_PATH = os.path.join(FINAL_DIR, 'skill_model.joblib')
TFIDF_PATH = os.path.join(FINAL_DIR, 'tfidf_skills.pkl')
CANDIDATE_PATH = os.path.join(FINAL_DIR, 'candidate_skills.json')
CATEGORIES_PATH = os.path.join(FINAL_DIR, 'categories.json')


_model = None
_tfidf = None
_candidate_skills = None
_categories = None
_cat_to_idx = None
_lower_to_cat = None


def skill_tokenizer(text):
    return [s.strip() for s in str(text).split(',') if s.strip()]


def load_artifacts():
    global _model, _tfidf, _candidate_skills, _categories, _cat_to_idx, _lower_to_cat
    if _model is not None:
        return
    import __main__
    if not hasattr(__main__, 'skill_tokenizer'):
        __main__.skill_tokenizer = skill_tokenizer
    _model = joblib.load(MODEL_PATH)
    _tfidf = joblib.load(TFIDF_PATH)
    with open(CANDIDATE_PATH, 'r', encoding='utf-8') as f:
```

```

        _candidate_skills = json.load(f)
    with open(CATEGORIES_PATH, 'r', encoding='utf-8') as f:
        _categories = json.load(f)
    _cat_to_idx = {c: i for i, c in enumerate(_categories)}
    _lower_to_cat = {c.lower(): c for c in _categories}

def predict_skills(user_skills, target_job, top_n=15):
    """Prediksi skill yang dibutuhkan + rekomendasi skill yang harus dipelajari (multi-label ML)."""
    load_artifacts()
    cat = _lower_to_cat.get(str(target_job).strip().lower())
    if cat is None:
        return None

    ctx = ', '.join(s.strip().lower() for s in user_skills)
    x_ctx = _tfidf.transform([ctx])

    n_cat = len(_categories)
    onehot = np.zeros((1, n_cat))
    onehot[0, _cat_to_idx[cat]] = 1.0
    X = hstack([x_ctx, csr_matrix(onehot)]).tocsr()

    probs = _model.predict_proba(X)[0]
    top_idx = np.argsort(-probs)[:top_n]
    required = [
        {'skill': _candidate_skills[j], 'prob': round(float(probs[j]), 4)}
        for j in top_idx
    ]

    user_norm = set(s.strip().lower() for s in user_skills)
    matched, gap = [], []
    for r in required:
        is_match = any(r['skill'] in u or u in r['skill'] for u in user_norm)
        if is_match:
            matched.append(r)
        else:

```

```

        gap.append(r)

    gap_score = len(gap) / len(required) * 100 if required else 0
    recommended = [dict(g, priority_rank=i + 1) for i, g in enumerate(gap)]

    return {
        'target_job': cat,
        'benchmark_count': len(required),
        'required': required,
        'matched': matched,
        'gap': gap,
        'recommended': recommended,
        'gap_score': round(gap_score, 1),
        'match_score': round(100 - gap_score, 1),
    }

```

Lampiran 4 Listing Program – CV Service / NLP Pipeline (cv_service.py)

File `cv_service.py` mengimplementasikan *pipeline* ekstraksi *skill* dari dokumen CV berformat PDF. *Pipeline* terdiri dari tiga lapisan:

1. ekstraksi teks menggunakan PyMuPDF (fitz)
2. pencocokan eksak dan *substring* terhadap daftar *skill* yang diketahui
3. ekstraksi *noun-chunk* menggunakan model NLP spaCy (`en_core_web_sm`) sebagai lapisan *fallback* tambahan.

```

import re
import io
import logging
from typing import List, Dict, Tuple

logger = logging.getLogger(__name__)

# — Optional heavy imports (graceful fallback if not installed yet)
try:
    import fitz  # PyMuPDF
    _FITZ_OK = True
except ImportError:
    _FITZ_OK = False
    logger.warning("PyMuPDF (fitz) not installed - PDF extraction unavailable.")

try:

```

```

import spacy

_SPACY_MODEL = None # loaded lazily
_SPACY_OK = True
except ImportError:
    _SPACY_OK = False
    logger.warning("spaCy not installed - NLP extraction disabled.")

# — spaCy lazy loader
def _get_nlp():
    global _SPACY_MODEL
    if not _SPACY_OK:
        return None
    if _SPACY_MODEL is None:
        try:
            _SPACY_MODEL = spacy.load("en_core_web_sm")
        except OSError:
            logger.warning(
                "spaCy model 'en_core_web_sm' not found. "
                "Run: python -m spacy download en_core_web_sm"
            )
    return _SPACY_MODEL

# — Text cleaning helpers
_NOISE = re.compile(r"^[^w\s\.\+\#\/\-]")
_WHITESPACE = re.compile(r"\s+")

def _clean(text: str) -> str:
    text = _NOISE.sub(" ", text)
    return _WHITESPACE.sub(" ", text).strip()

def _tokenise(text: str) -> List[str]:
    """Split on common CV delimiters and return lowercase tokens."""
    tokens = re.split(r"[\n,;|••▶▶\t]+", text)
    result = []
    for t in tokens:

```

```

        t = t.strip().lower()
        if 2 <= len(t) <= 60:
            result.append(t)
    return result

# — PDF extraction
def extract_text_from_pdf(file_bytes: bytes) -> str:
    """Return concatenated plain text from all pages of a PDF."""
    if not _FITZ_OK:
        raise RuntimeError("PyMuPDF is not installed. Run: pip install PyMuPDF")

    try:
        doc = fitz.open(stream=file_bytes, filetype="pdf")
        pages_text = []
        for page in doc:
            pages_text.append(page.get_text("text"))
        doc.close()
        return "\n".join(pages_text)
    except Exception as exc:
        raise ValueError(f"Could not parse PDF: {exc}") from exc

# — Skill extraction
def _build_known_index(known_skills: List[str]) -> Dict[str, str]:
    """Build a normalised lowercase → original mapping for fast lookup."""
    index: Dict[str, str] = {}
    for skill in known_skills:
        norm = skill.strip().lower()
        index[norm] = skill
    return index

def _exact_and_substring_match(
    tokens: List[str],
    known_index: Dict[str, str],
) -> Tuple[List[str], set]:
    """Layer 1 & 2: exact match + substring containment."""

```

```

found: Dict[str, str] = {} # norm → original
matched_knowns: set = set()

for token in tokens:
    # exact
    if token in known_index:
        found[token] = known_index[token]
        matched_knowns.add(token)
        continue

    # substring: token contained in a known skill or vice versa
    for known_norm, known_orig in known_index.items():
        if known_norm in matched_knowns:
            continue
        if (
            (len(token) >= 3 and token in known_norm)
            or (len(known_norm) >= 3 and known_norm in token)
        ):
            found[known_norm] = known_orig
            matched_knowns.add(known_norm)
            break

return list(found.values()), matched_knowns

def _spacy_noun_chunk_match(
    text: str,
    known_index: Dict[str, str],
    already_found: set,
) -> List[str]:
    """Layer 3: extract noun-chunks via spaCy and match against known skills."""
    nlp = _get_nlp()
    if nlp is None:
        return []

    extra: Dict[str, str] = {}
    doc = nlp(text[:50_000]) # cap for performance

```



```

for chunk in doc.noun_chunks:
    chunk_text = chunk.text.strip().lower()
    chunk_text = re.sub(r"\s+", " ", chunk_text)

    if chunk_text in known_index and chunk_text not in already_found:
        extra[chunk_text] = known_index[chunk_text]
        continue

# partial overlap with known skills (for multi-word phrases)
for known_norm, known_orig in known_index.items():
    if known_norm in already_found or known_norm in extra:
        continue
    if len(known_norm.split()) > 1:
        if known_norm in chunk_text or chunk_text in known_norm:
            extra[known_norm] = known_orig
            break

return list(extra.values())

def extract_skills_from_text(
    text: str,
    known_skills: List[str],
    max_skills: int = 60,
) -> Dict:
    """
    Main NLP pipeline.

    Returns:
    {
        "detected_skills": [...],      # cleaned list ready for /detect-gap
        "skill_count": N,
        "raw_text_preview": "...",     # first 500 chars for UI preview
        "extraction_notes": "...",     # info about pipeline used
    }
    """
    if not text or not text.strip():

```

```

    return {
        "detected_skills": [],
        "skill_count": 0,
        "raw_text_preview": "",
        "extraction_notes": "No text extracted from PDF.",
    }

cleaned_text = _clean(text)
known_index = _build_known_index(known_skills)
tokens = _tokenise(cleaned_text)

# Layer 1 + 2: exact & substring
skills_l1, matched_set = _exact_and_substring_match(tokens, known_index)

# Layer 3: spaCy noun chunks (additive)
skills_l3 = _spacy_noun_chunk_match(cleaned_text, known_index, matched_set)

# Merge, deduplicate, cap
all_skills_norm: Dict[str, str] = {}
for s in skills_l1 + skills_l3:
    norm = s.strip().lower()
    all_skills_norm[norm] = s

final_skills = list(all_skills_norm.values())[:max_skills]

notes = "Keyword matching"
if _SPACY_OK and _get_nlp() is not None:
    notes = "Keyword matching + spaCy NLP (en_core_web_sm)"
elif not _SPACY_OK:
    notes = "Keyword matching only (spaCy not installed)"

return {
    "detected_skills": final_skills,
    "skill_count": len(final_skills),
    "raw_text_preview": cleaned_text[:500],
    "extraction_notes": notes,
}

```

Lampiran 5 Listing Program – Data Service (data_service.py)

File `data_service.py` bertugas memuat *dataset* pekerjaan dari file CSV (`JobsDatasetProcessed.csv`) dan membangun indeks *skill* per kategori pekerjaan yang digunakan sebagai *benchmark* pada proses deteksi kesenjangan kompetensi. Setiap kategori pekerjaan dipetakan ke kamus *skill* beserta frekuensi kemunculannya.

```
import pandas as pd

import numpy as np

from collections import defaultdict

import os

DATA_DIR = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(__file__))),
                        'use_dataset')

JOBS_CSV = os.path.join(DATA_DIR, 'JobsDatasetProcessed.csv')

def load_jobs_data():
    df_jobs = pd.read_csv(JOBS_CSV)
    return df_jobs

def build_job_category_skills(df_jobs):
    job_category_skills = defaultdict(lambda: defaultdict(int))
    for _, row in df_jobs.iterrows():
        query = str(row.get('Query', '')).strip().lower()
        skills = str(row.get('IT Skills', '')).strip()
        if not skills or skills == 'nan':
            continue
        for s in skills.split(','):
            cleaned = s.strip().lower()
            if cleaned:
                job_category_skills[query][cleaned] += 1
    return {cat: dict(skills) for cat, skills in job_category_skills.items()}

def get_available_categories(job_category_skills):
    return sorted(job_category_skills.keys())
```

Lampiran 6 Output Aplikasi Elvit



Gambar 1. Tampilan Beranda ElvIT

SELF ASSESSMENT

Evaluasi kemampuanmu sendiri

Isi kuesioner ini dengan jujur. Hasilnya akan dibandingkan dengan kebutuhan riil industri berdasarkan analisis ribuan data lowongan kerja.

Nilai level kemampuanmu
Pilih subdomain dan nilai secara jujur

[Software Development](#) [Data Engineering](#) [Cybersecurity](#) [Cloud Computing](#)

JavaScript Pemrograman frontend & backend dasar	Tidak Tahu Dasar Menengah Mahir
TypeScript Static typing untuk JS skala besar	Tidak Tahu Dasar Menengah Mahir
React Membangun UI berbasis komponen	Tidak Tahu Dasar Menengah Mahir
Node.js Runtime backend JavaScript	Tidak Tahu Dasar Menengah Mahir
Git Version control system	Tidak Tahu Dasar Menengah Mahir

0 DARI 10 SKILL DINILAI

[Lihat Hasil Analisis ->](#)

Gambar 2. Halaman *Self Assessment*



[Beranda](#)
[Asesmen](#)
[Manual Check](#)
[Tentang](#)
[Scan CV](#)

Nilai level kemampuanmu

Pilih subdomain dan nilai secara jujur

Software Development
Data Engineering
Cybersecurity
Cloud Computing

JavaScript
 Pemrograman frontend & backend dasar

Tidak Tahu
Dasar
Menengah
Mahir

TypeScript
 Static typing untuk JS skala besar

Tidak Tahu
Dasar
Menengah
Mahir

React
 Membangun UI berbasis komponen

Tidak Tahu
Dasar
Menengah
Mahir

Node.js
 Runtime backend JavaScript

Tidak Tahu
Dasar
Menengah
Mahir

Git
 Version control system

Tidak Tahu
Dasar
Menengah
Mahir

5 DARI 16 SKILL DINILAI

Lihat Hasil Analisis -->

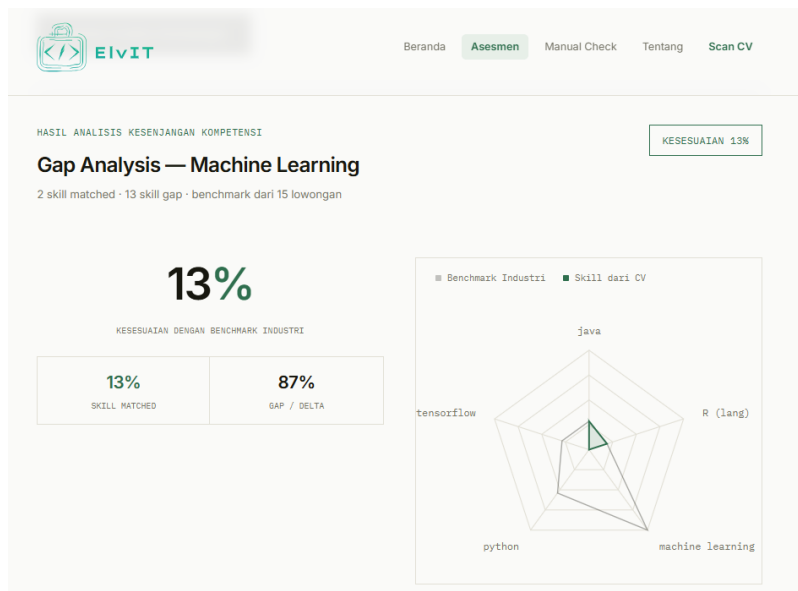
Tahap 2 Pilih Target Pekerjaan

Bandingkan skill yang baru saja kamu nilai dengan standar untuk posisi:

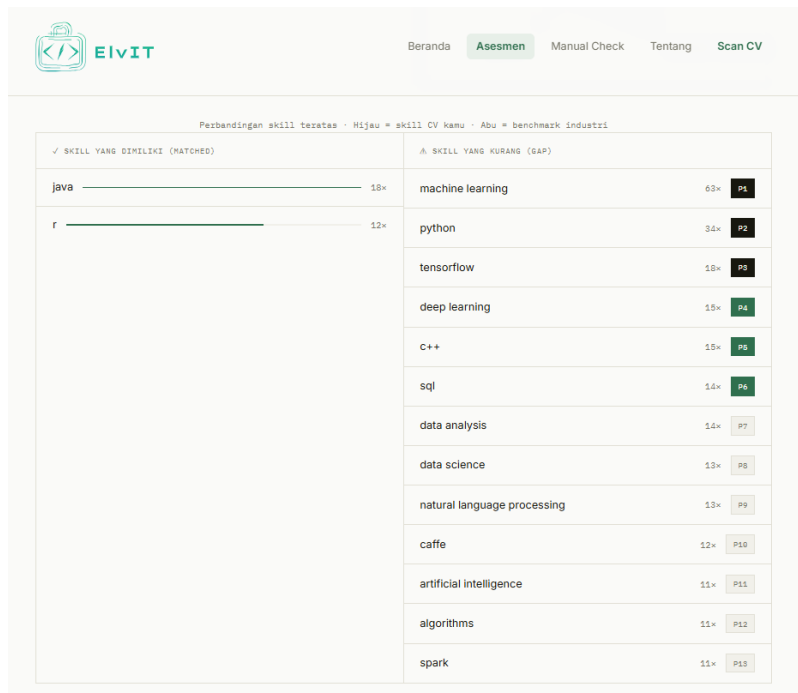
Machine Learning

Jalankan Analisis Kesenjangan

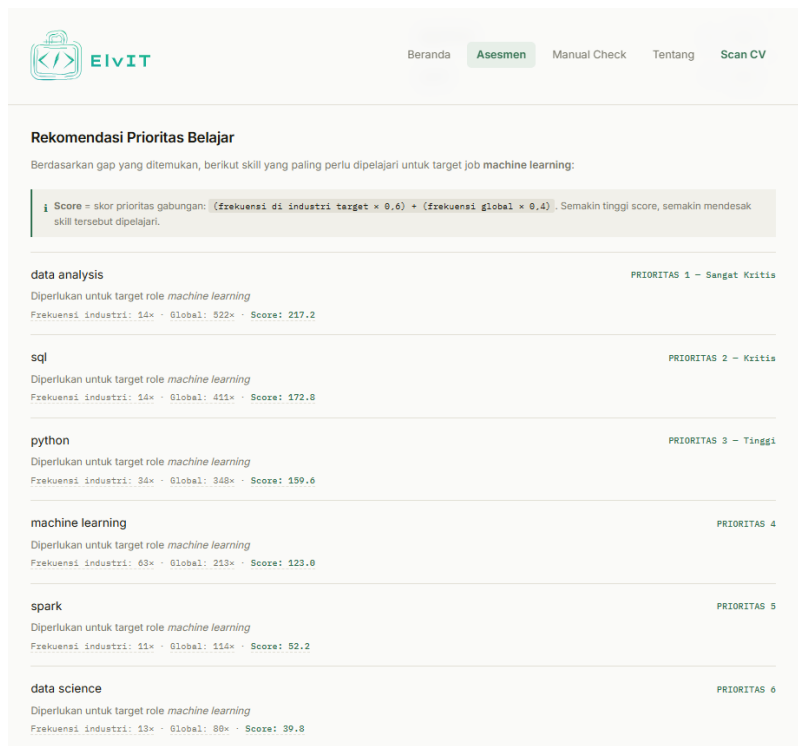
Gambar 3. *Input Self Assessment*



Gambar 4. *Output Gap Self Assessment*



Gambar 5. *Output Perbandingan Skill Self Assessment*



Gambar 6. *Output Rekomendasi Belajar Self Assessment*

CEK KESENJANGAN SKILL MANUAL

Input Skill Secara Manual

Ketikkan skill yang kamu miliki satu per satu, lalu pilih target pekerjaan untuk melihat kesenjangan kompetensimu.

Tahap 1 Masukkan Skill

Ketik skill dan tekan Enter atau klik Tambah.

Contoh: Python, React, Data Analysis

Tambah

Gambar 7. Halaman Manual Check

CEK KESENJANGAN SKILL MANUAL

Input Skill Secara Manual

Ketikkan skill yang kamu miliki satu per satu, lalu pilih target pekerjaan untuk melihat kesenjangan kompetensimu.

Tahap 1 Masukkan Skill

Ketik skill dan tekan Enter atau klik Tambah.

Contoh: Python, React, Data Analysis

Tambah

1 skill ditambahkan

Python

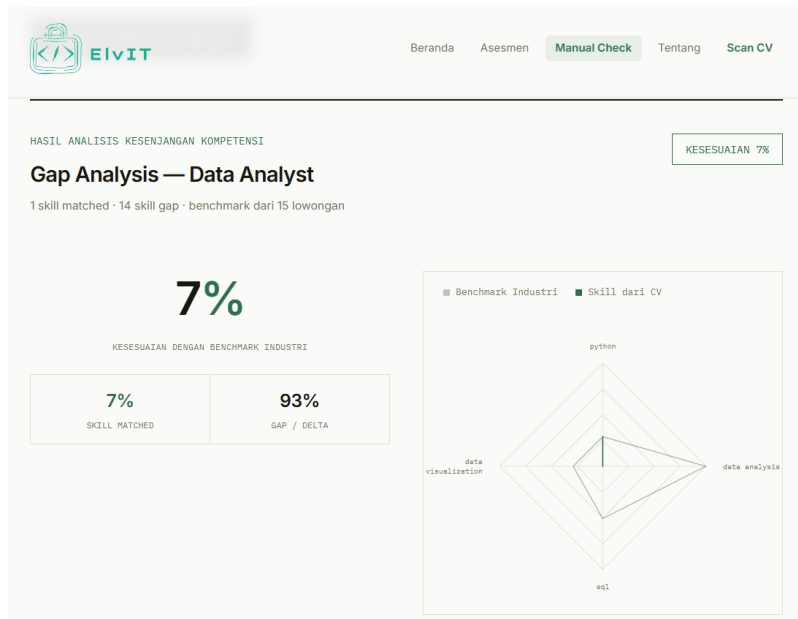
Tahap 2 Pilih Target Pekerjaan

Bandingkan skill yang kamu miliki dengan standar untuk posisi:

Data Analyst

Jalankan Analisis Kesenjangan

Gambar 8. *Input* Manual Check



Gambar 9. *Output Gap Manual Check*

Perbandingan skill teratas · Hijau = skill CV kamu · Abu = benchmark industri

✓ SKILL YANG DIMILIKI (MATCHED)	△ SKILL YANG KURANG (GAP)
python ————— 21x	data analysis 73x P1
	sql 37x P2
	data visualization 21x P3
	excel 19x P4
	database management 19x P5
	r 18x P6
	statistical analysis 16x P7
	tableau 13x P8
	data mining 12x P9
	data modeling 12x P10
	data management 11x P11
	data manipulation 11x P12

Gambar 10. *Output Perbandingan Skill Manual Check*



[Beranda](#)
[Asesmen](#)
[Manual Check](#)
[Tentang](#)
[Scan CV](#)

Rekomendasi Prioritas Belajar

Berdasarkan gap yang ditemukan, berikut skill yang paling perlu dipelajari untuk target job **data analyst**:

Score = skor prioritas gabungan: $(\text{frekuensi di industri target} \times 0,6) + (\text{frekuensi global} \times 0,4)$. Semakin tinggi score, semakin mendesak skill tersebut dipelajari.

data analysis Diperlukan untuk target role <i>data analyst</i> Frekuensi industri: 73% • Global: 822% • Score: 252.6	PRIORITAS 1 – Sangat Kritis
sql Diperlukan untuk target role <i>data analyst</i> Frekuensi industri: 37% • Global: 411% • Score: 186.6	PRIORITAS 2 – Kritis
data visualization Diperlukan untuk target role <i>data analyst</i> Frekuensi industri: 21% • Global: 163% • Score: 77.8	PRIORITAS 3 – Tinggi
excel Diperlukan untuk target role <i>data analyst</i> Frekuensi industri: 19% • Global: 161% • Score: 75.8	PRIORITAS 4
data modeling Diperlukan untuk target role <i>data analyst</i> Frekuensi industri: 12% • Global: 164% • Score: 72.8	PRIORITAS 5

Gambar 11. *Output* Rekomendasi Belajar Manual Check



[Beranda](#)
[Asesmen](#)
[Manual Check](#)
[Tentang](#)
[Scan CV](#)

SCAN CV – DETEKSI KESENJANGAN KOMPETENSI IT

Upload CV-mu, ketahui gap-nya — otomatis.

Ekstraksi skill dari PDF CV menggunakan NLP, lalu dibandingkan langsung dengan benchmark lowongan kerja nyata. Pilih target job, hasil analisis muncul dalam hitungan detik.

Upload CV

Parse & Ekstraksi

Pilih Target Job

Deteksi Gap

Hasil & Rekomendasi

01 Upload CV (PDF)

Drag & drop CV PDF

atau klik untuk pilih file

PDF • Maksimal 10 MB

RINGKASAN PLATFORM

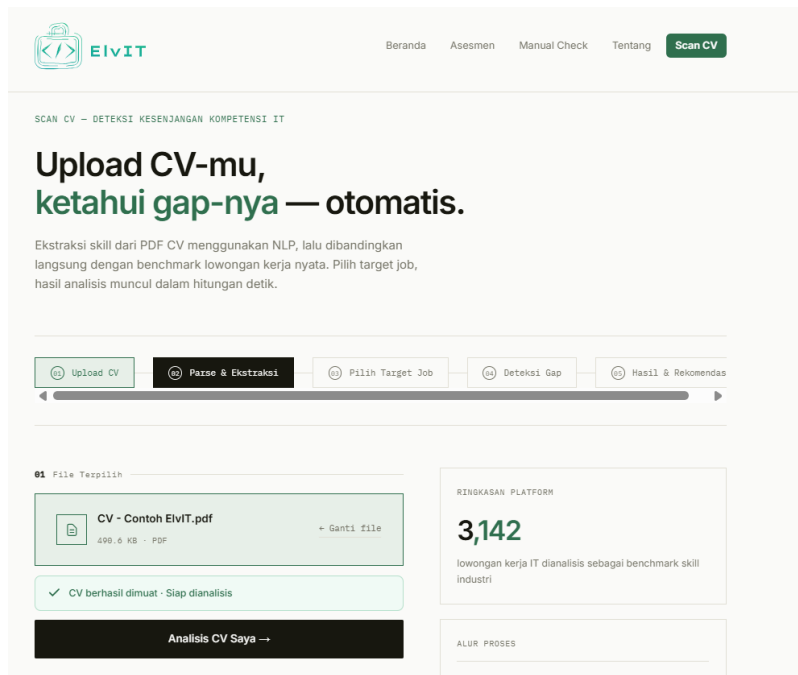
3,142

lowongan kerja IT dianalisis sebagai benchmark skill industri

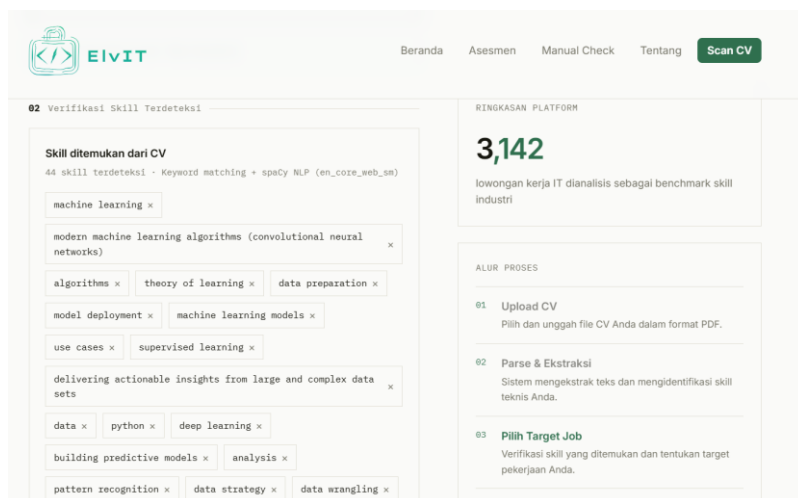
ALUR PROSES

01 Upload CV

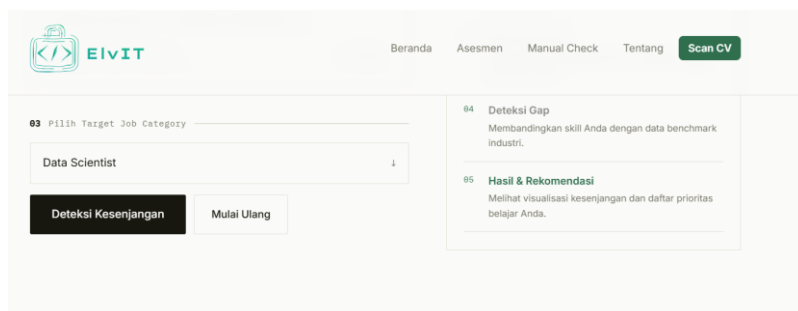
Gambar 12. Halaman Scan CV



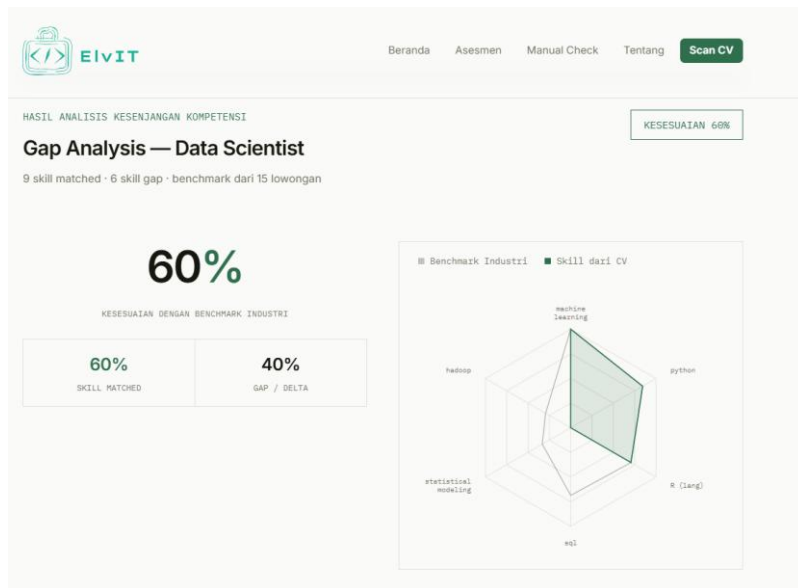
Gambar 13. *Input CV*



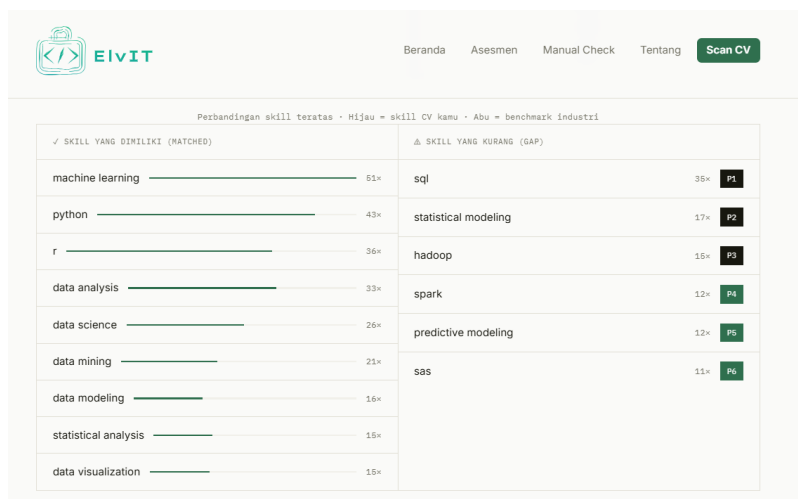
Gambar 14. *Output Skill yang Dimiliki*



Gambar 15. *Input Kategori Pekerjaan*



Gambar 16. *Output Gap Scan CV*



Gambar 17. *Output Perbandingan Skill Scan CV*



[Beranda](#)
[Asesmen](#)
[Manual Check](#)
[Tentang](#)
[Scan CV](#)

Rekomendasi Prioritas Belajar

Berdasarkan gap yang ditemukan, berikut skill yang paling perlu dipelajari untuk target job *data scientist*:

i Score = skor prioritas gabungan: $(\text{frekuensi di industri target} \times 0,6) + (\text{frekuensi global} \times 0,4)$. Semakin tinggi score, semakin mendesak skill tersebut dipelajari.

sql Diperlukan untuk target role <i>data scientist</i> Frekuensi industri: 35* · Global: 411* · Score: 188.4	PRIORITAS 1 – Sangat Kritis
spark Diperlukan untuk target role <i>data scientist</i> Frekuensi industri: 12* · Global: 114* · Score: 52.8	PRIORITAS 2 – Kritis
hadoop Diperlukan untuk target role <i>data scientist</i> Frekuensi industri: 15* · Global: 187* · Score: 51.8	PRIORITAS 3 – Tinggi
sas Diperlukan untuk target role <i>data scientist</i> Frekuensi industri: 11* · Global: 68* · Score: 33.8	PRIORITAS 4
statistical modeling Diperlukan untuk target role <i>data scientist</i>	PRIORITAS 5

Gambar 18. *Output* Rekomendasi Belajar Scan CV



[Beranda](#)
[Asesmen](#)
[Manual Check](#)
[Tentang](#)
[Scan CV](#)

TENTANG PROYEK

Riset: Deteksi Kesenjangan Kompetensi IT dengan Pendekatan Hybrid NLP

Aplikasi ElviT dibangun sebagai prototipe hasil penelitian untuk mendeteksi gap antara kompetensi yang dimiliki individu dengan kebutuhan industri secara otomatis.

Gambar 19. Tampilan Halaman Tentang ElviT